Blockchain-Based Remittance System

Prepared By: Abdallah Namoura

Advisor: Ms. Hana' Hirbawi

Supervisor: Miroslav Vegh

Company: Diligio

Al-Quds University

Faculty of Dual Studies

Department of Information Technology (IT)

Year 2025/2026

Table of Contents

1. Introduction

1.1. Executive Summary:

Traditional remittance systems such as banks and services like Western Union are expensive, slow, and centralized. International money transfers often take days to complete and impose high transaction fees, which is especially harmful for migrant workers and small businesses.

This project proposes a web-based *Blockchain-Based Remittance System* that allows users to send and receive money securely using Ethereum smart contracts. The system eliminates middlemen, reduces transaction cost, and ensures transparency by recording all payments on the blockchain.

The web-based platform allows:

- Sending and receiving funds using crypto
- Wallet integration
- Checking balances in crypto and simulated fiat
- Viewing transaction history
- Secure transfers through smart contracts

The project demonstrates how blockchain technology can modernize financial transactions and increase user trust.

1.2. Audience

This document is written to ensure a common understanding of the system's requirements, scope, and technical design for the following:

- Evaluators and supervisors: to detail the project's objectives, features, scope, and architectural choices for evaluation and approval of the graduation project.
- Project stakeholders and potential investors: to demonstrate the project's key features, market relevance, and financial advantages (i.e., low-cost remittance), potentially securing future funding or support.
- Future maintainers and researchers: to provide comprehensive documentation of the system's design and technical approach, ensuring clarity for any future updates, maintenance, or academic research.
- End users and administrators: to define the system's interface and capabilities, focusing on the User Requirements and guiding the creation of the User Manual.

2. Research and Findings

2.1. Related Topics

2.1.1. Research Papers

2.1.1.1. Owolabi, Omoshola S., et al. (2024). Blockchain-Based System for Secure and Efficient Cross-Border Remittances: A Potential Alternative to SWIFT.

This paper proposes a blockchain-based remittance system as an alternative to SWIFT, arguing that traditional international transfers are slow, costly, and centralized. Their system uses smart contracts to automate payments and record all transactions on a distributed ledger, improving transparency and security. The authors show that blockchain can significantly reduce transfer time and cost while increasing system reliability. They also highlight its potential to improve financial inclusion in underserved regions, while noting challenges such as regulation and scalability.

2.1.1.2. Soufaih, A. (2020). Revolutionizing international remittance payments using cryptocurrency and blockchain-based technology.

Soufaih (2020) argues that cryptocurrency and blockchain technology can transform international remittance systems by removing intermediaries, lowering fees, and increasing transaction speed. The paper explains how blockchain enables peer-to-peer transfers using decentralized networks, making international payments faster and more efficient. The author highlights how crypto-based remittances can improve financial inclusion by giving unbanked users access through mobile devices. Challenges such as regulation, price volatility, and public adoption are also discussed as barriers to widespread use.

2.1.2. Related Systems

2.1.2.1. Chipper – Chipper Cash

Figure 1: Chipper Cash Logo

Chipper (see Figure 1) is a mobile fintech platform for fast, free, and low-cost cross-border money transfers across Africa and to the U.S., enabling P2P

payments, virtual Visa cards for global online spending, bill payments, airtime purchases, and even investing in stocks, aiming to boost financial inclusion with secure, easy-to-use services for individuals and businesses.

Key Features:

- Offers secure, efficient currency exchange and international payments for businesses.
- Provides cutting-edge technology for both wholesale and retail currency needs.
- Helps companies manage currency risk with competitive rates and hedging strategies.
- Specializes in frontier markets, facilitating trade across Africa.
- Serves multinational corporations, remittance providers, and enterprises.

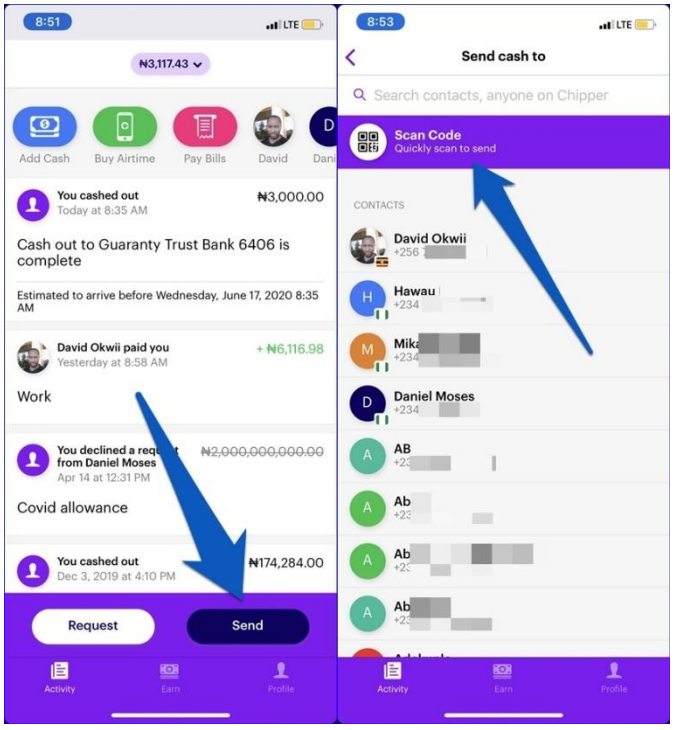


Figure 2: Sending cash in Chipper Cash

2.1.2.2. AZA Finance

Figure 3: AZA Finance Logo

AZA Finance (see Figure 2) is a leading pan-African fintech providing cross-border payments, foreign exchange, and treasury services, founded in 2013 enabling businesses globally to transact in African and G20 currencies efficiently through its API and web platforms, reducing costs and increasing payment speed to, from, and within frontier markets. It connects businesses, banks, and payment providers to Africa's growing markets, acting as a key infrastructure for global trade.

Key Features:

- Focuses on eliminating high costs for cross-border transactions.
- Aims to serve underbanked populations by simplifying access to digital finance.
- Uses encryption to protect user data and payments.
- Offers payment solutions for businesses to accept online payments.

2.2. User Analysis (Personas)

The personas are created to supply a deeper insight into the target users' personality, needs, preferences and routine. These personas allow us to serve as a reference to help us create a user-centric system that matches the challenges, expectations and needs of the target users.

2.2.1. Persona 1:

Figure 4: User Persona 1 (Ahmad Khalil)

2.2.2. Persona 2:

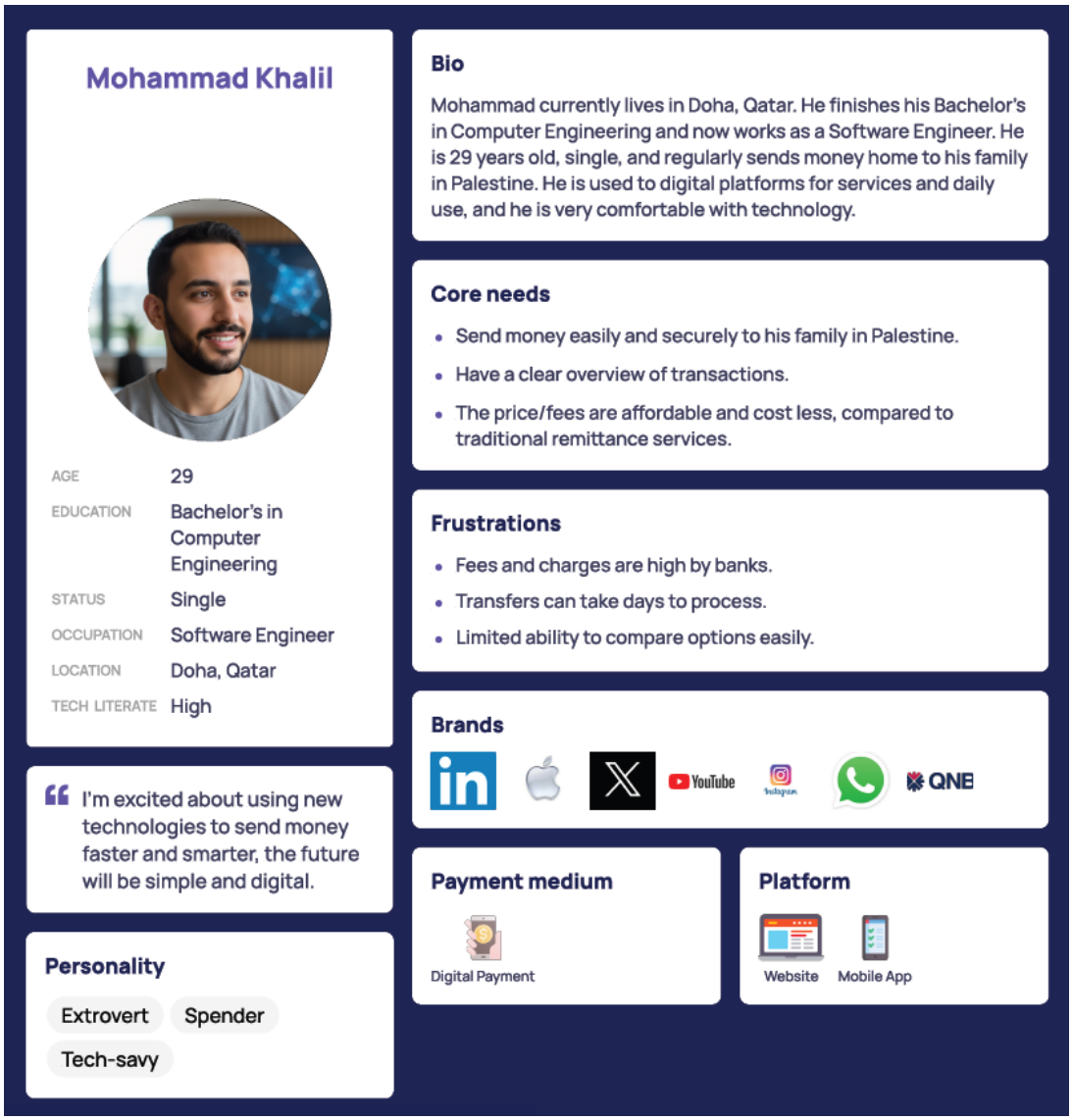


Figure 5: User Persona 2 (Mohammad Khalil)

3. Feasibility Study and Proposed Solution

Several existing methods are used today to perform international remittance. Each has limitations that affect usability, cost, and performance.

3.1. Alternative Solutions

- **Banks and Financial Institutions:**
  Traditional banks use correspondent banking networks for international transfers. These systems involve multiple intermediaries, leading to long transaction times and high service fees.
- **Western Union and Similar Services:**
  Money transfer companies enable faster payments than banks, but they impose expensive transfer fees and often require physical branch visits.
- **Online Payment Platforms (PayPal, Wise, Skrill):**
  These services offer digital transfers but are limited by country restrictions, foreign exchange fees, and account verification barriers.

3.2. Proposed Solution

The proposed system is a **Blockchain-Based Remittance System** that enables users to perform international money transfers securely and efficiently through smart contracts.

The system provides digital wallets for users, enabling them to send and receive funds directly using blockchain technology. All transactions are recorded transparently on the blockchain and verified through consensus mechanisms. The system will include:

- User authentication and wallet integration
- Transaction history tracking
- Simulated crypto-to-fiat display
- Secure transaction validation
- Web-based user interface

This solution reduces dependency on banks and intermediaries, lowers transaction costs, improves speed, and increases accessibility for users in underserved regions. The design ensures scalability, simplicity, and transparency.

4. Technical Approach

4.1. Tools

4.1.1. Development Tools

4.1.1.1. Frontend:

- React.js
- HTML/CSS
- MetaMask

4.1.1.2. Backend:

- Node.js
- Express.js
- Blockchain
- Ethereum: The system uses a smart contract deployed on the Ethereum blockchain to validate transactions and execute cryptocurrency transfers. The smart contract only handles transaction execution and does not store user information or personal data.

4.1.2. Version Control Tools

- GitHub
- Postman
- Figma

4.2. Database

MongoDB will be used as the main database, to store user profiles, transaction history, logs and simulated fiat values

4.3. Environment

The development and execution environment includes:

- Web Browser: Google Chrome / Vivaldi
- Blockchain Network: Ethereum
- Runtime: Node.js
- IDE: VSCode

5. Final SRS

5.1. Requirements

5.1.1. Functional Requirements

1. Core Features

1.1. The system shall allow a user to register using a unique email address and a password containing at least 8 characters, including at least one uppercase letter, one number, and one special character.

1.2. The system shall reject registration when the email address is already registered or does not match a valid email format.

1.3. The system shall allow registered users to log in using their email address and password.

1.4. The system shall block login attempts for disabled accounts and display an account-disabled message.

1.5. The system shall allow users to reset their password by requesting a password-reset link sent to their registered email address. The reset link shall expire after 15 minutes and may be used only once.

1.6. The system shall detect whether the MetaMask wallet provider is available before starting the wallet-connection process.

1.7. The system shall link a wallet address to the logged-in user only after the user approves the MetaMask request and successfully completes wallet-ownership signature verification, and the system shall display a confirmation message indicating that the wallet has been successfully verified and linked to the user account.

1.8. The system shall display the user's current Ethereum (ETH) wallet balance on the main dashboard and refresh the displayed value automatically every 15 seconds.

1.9. The system shall display a simulated fiat equivalent of the ETH wallet balance using a configurable exchange rate on the main dashboard, refresh the displayed value automatically every 15 seconds together with the ETH balance.

1.10. The system shall allow a user to disconnect a previously connected wallet from their account without affecting stored transaction records.

1.11. The system shall maintain an association between each registered user account and its linked wallet address by requiring user authentication before linking, verifying wallet ownership through a MetaMask signature request, and preventing modification of the linked wallet by unauthorized users.

2. User Interaction

2.1. The system shall display an error message that identifies the specific cause of failure when the wallet provider is missing, the connection request is rejected, or wallet-ownership verification fails.

2.2. The system shall allow a user to enter either a valid wallet address or a registered user identifier as the transfer receiver.

2.3. The system shall display a transaction-summary screen showing the receiver and total amount before blockchain execution.

2.4. The system shall allow the user to confirm or cancel the transfer from the transaction-summary screen.

2.5. When the user cancels a transfer before confirmation, the system shall not submit a blockchain transaction.

2.6. The system shall allow users to view their complete transaction history.

2.7. The system shall allow users to filter transaction history by date range, transaction type (sent or received), and status (pending, successful, or failed).

2.8. The system shall display “No transactions found” when the selected transaction-history filters produce no matching records.

2.9. The system shall display red inline validation messages directly below any field that fails validation.

2.10. The system shall display a notification message within 2 seconds after account registration, login, wallet connection, password reset, or transaction completion. The notification shall indicate whether the operation was successful or unsuccessful and shall include a specific reason when the operation fails.

2.11. The system shall display loading indicators while retrieving wallet balances, transaction history, or blockchain transaction statuses.

3. Business Rules

3.1. The system shall prevent users from proceeding to the next step or submitting a transaction when required fields are missing, the amount is zero or negative, the wallet address is invalid, or the amount exceeds the user’s available balance. The submission or confirmation button shall remain disabled until all validation requirements are satisfied.

3.2. The system shall prevent duplicate transaction submissions by disabling repeated transfer requests until the current transaction is completed or cancelled and by rejecting any identical transaction request received during processing.

3.3. The system shall allow transfers only when the sender has a verified and connected wallet.

3.4. The system shall prevent users from transferring funds to their own wallet address.

3.5. The system shall enforce role-based access control for administrative functions.

3.6. The system shall restrict access to transaction history and wallet information to authenticated users only.

3.7. The system shall validate all user inputs before processing any account, wallet, or transaction operation.

4. Data Processing

4.1. The system shall record the sender wallet, receiver wallet, timestamp, amount, status, transaction hash, and failure reason when applicable.

4.2. The system shall update the stored transaction status immediately after receiving the blockchain execution result and shall synchronize the transaction record in the database within 2 seconds of receiving the result.

4.3. The system shall log critical events, including successful login, failed login, wallet-connection attempts, transfer attempts, transfer results, and admin actions.

4.4. The system shall store transaction records in the database and preserve them for future retrieval and auditing.

4.5. The system shall synchronize transaction information between the blockchain network and the database to maintain consistency.

4.6. The system shall record the date and time of all account, wallet, and transaction activities using a standardized timestamp format.

4.7. The system shall update wallet balance information whenever a successful transaction is confirmed on the blockchain.

4.8. The system shall retain failed transaction records together with the associated failure reason for future reference and analysis.

5. Admin

5.1. The system shall restrict admin functions to authenticated users with the administrator role.

5.2. The administrator shall be able to monitor system transaction records and their status.

5.3. The administrator shall be able to view user accounts and disable an account.

5.1.2. Non-Functional Requirements

1. **Usability**

1.1. The system shall provide a consistent user interface across all pages by using the same navigation structure, button styles, form layouts, colour scheme, and typography for equivalent functions.

1.2. The system shall allow authenticated users to access the dashboard, wallet connection, fund transfer, and transaction history functions directly from the main navigation menu with no more than one navigation action from the dashboard.

1.3. The system shall display descriptive labels for all input fields and provide context-specific instructions where user input requires a defined format or action.

1.4. During usability testing, at least 80% of test participants who have not previously used the system shall be able to successfully connect a wallet, send funds, and view transaction history without assistance or external documentation.

2. **Performance**

2.1. The system shall respond to user requests within 2 seconds under normal operating conditions, excluding blockchain confirmation time.

2.2. Average page load time shall not exceed 2 seconds under normal operating conditions.

2.3. The system shall validate and submit a transaction request within 2 seconds after user confirmation, excluding blockchain confirmation time.

2.4. The application shall display the blockchain transaction result, including the transaction hash or failure reason, within 2 seconds after receiving the response from the blockchain network.

3. **Scalability**

The system architecture shall support the addition of new features and modules without requiring major changes to existing functionality.

4. **Maintainability**

4.1. The frontend, backend, blockchain, and database logic shall be implemented as separate modules with clearly defined responsibilities.

4.2. The system shall use reusable components, consistent naming conventions, and organized code structures to support maintenance and future enhancements.

5. **Reliability**

5.1. The system shall maintain consistency between blockchain transaction records and MongoDB transaction records.

5.2. The system shall preserve transaction records during temporary application, network, or database failures.

5.3. The system shall reconcile transaction records using the sender wallet, receiver wallet, amount, transaction hash, and transaction status.

5.4. The system shall update transaction records with the final blockchain status after recovery from temporary failures where reconciliation is possible.

5.1.3. User Requirements

1. Account Management

1.1. Users shall be able to register an account using a unique email address and a password containing at least 8 characters, including at least one uppercase letter, one number, and one special character

1.2. Users shall be able to log in using their registered email address and password.

1.3. Users shall be able to recover their password through a secure password-reset process that verifies account ownership before allowing a new password to be set.

2. Wallet Management

2.1. Users shall be able to connect a wallet to their account when the provider is available.

2.2. Users shall be able to disconnect a previously connected wallet from their account.

2.3. Users shall be notified when no wallet is connected, when the provider is unavailable, when a connection request is rejected, or when wallet connection fails.

2.4. Users shall complete wallet-ownership verification through a signed message before a wallet is linked to their account.

2.5. Users shall be able to view the connection status of their linked wallet.

3. Transaction Operations

3.1. Users shall be able to send cryptocurrency to a valid recipient wallet address or a registered user associated with a valid wallet address.

3.2. Users shall be able to review transaction details, including recipient and transfer amount, on a transaction-summary screen before submission

3.3. Users shall be able to confirm or cancel a transaction from the transaction-summary screen before blockchain submission.

3.4. Users shall be able to view the status of submitted transactions, including pending, successful, and failed transactions.

4. Information Access

4.1. Users shall be able to view their Ethereum (ETH) wallet balance on the main dashboard.

4.2. Users shall be able to view a simulated fiat equivalent of their ETH balance using a configurable exchange rate.

4.3. Users shall be able to view their complete transaction history.

4.4. Users shall be able to filter transaction history by date range, transaction type (sent or received), and transaction status (pending, successful, or failed).

5.1.4. System Requirements

1. The system shall enforce user authentication before granting access to protected functions.
2. The system shall process cryptocurrency transfer requests through Ethereum smart contracts.
3. The system shall store user, wallet, transaction, and system log data persistently in MongoDB.
4. The system shall be accessible through modern web browsers that support JavaScript and wallet provider integration.

5.1.5. Dependability / Security Requirements

The requirements in this section are aligned with the applicable practices of the NIST Secure Software Development Framework (SSDF) Version 1.1. The degree of implementation shall be proportional to the system risks, academic project scope, technical feasibility, and selected deployment environment.

1. Sensitive Data and Security Scope

1.1. The system shall treat the following information as sensitive data:

- user email addresses and account information
- password hashes
- password-reset tokens
- authentication tokens and session identifiers
- linked wallet addresses
- wallet-ownership verification records
- transaction records
- administrative actions
- audit logs

- database backups
- API credentials, database credentials, environment variables, and deployment secrets

1.2. The system shall not request, collect, process, transmit, log, or store blockchain private keys, wallet seed phrases, wallet recovery phrases, or wallet recovery credentials under any circumstances.

1.3. The system shall document the following trust boundaries:

- user browser and React frontend
- Node.js and Express backend API
- MongoDB database
- MetaMask wallet provider
- Ethereum blockchain network and smart contract
- password-reset email delivery service
- source-code repository and deployment environment

1.4. Security requirements and mitigation decisions shall be reviewed when a major system feature is added, when the architecture changes, when a significant dependency is updated, when a vulnerability is identified, or before a production release.

1.5. Any approved exception to a security requirement shall be documented with its justification, affected component, potential impact, temporary mitigation, responsible person, and review date.

2. Authentication and Account Protection

2.1. The system shall require user authentication before granting access to protected functions, including wallet linking, wallet disconnection, balance viewing, fund transfers, transaction history, profile modification, and administrative functions.

2.2. The backend shall independently enforce authentication and authorization checks for every protected API endpoint.

2.3. The system shall not treat hidden buttons, disabled interface elements, or inaccessible frontend routes as sufficient authorization controls.

2.4. The system shall apply RBAC to administrative functions.

2.5. Administrative access shall be denied by default unless the authenticated account has an explicitly authorized administrator role.

2.6. User passwords shall be stored only as salted hashes generated using an industry-standard adaptive password-hashing algorithm.

2.7. The system shall never store, log, display, or transmit a user password in plaintext after the password has been submitted for processing.

2.8. The system shall enforce the same password rules during registration and password reset. A password shall contain at least 8 characters, including at least one uppercase letter, one number, and one special character.

2.9. Password-reset links shall expire after 15 minutes and shall be usable only once.

2.10. A previously issued password-reset link shall become invalid after successful use or after a newer password-reset link is requested.

2.11. Password-reset tokens shall be generated using a cryptographically secure random mechanism.

2.12. Password-reset tokens shall not be stored in plaintext.

2.13. The system shall invalidate active sessions when the user logs out, when the user successfully resets the password, or when an administrator disables the account.

2.14. The system shall reject login attempts for disabled accounts and display an account-disabled message.

2.15. The system shall limit repeated failed login attempts by applying a temporary delay, temporary lockout, or equivalent protective control after a configurable threshold is reached.

2.16. Authentication failure messages shall avoid revealing whether a specific email address is registered unless the business process explicitly requires that information.

2.17. When cookies are used for authentication, session cookies shall use the Secure, HttpOnly, and appropriate SameSite attributes.

3. Wallet and Blockchain Security

3.1. The system shall detect whether the MetaMask wallet provider is available before initiating the wallet-connection process.

3.2. The system shall link a wallet address to a user account only after the authenticated user approves the MetaMask connection request and successfully completes wallet-ownership verification.

3.3. Wallet-ownership verification shall use a signed challenge that includes a unique nonce, an expiration time, and a reference to the authenticated user account or session.

3.4. Each wallet-ownership verification challenge shall be usable only once to prevent replay attacks.

3.5. The backend shall verify the signed wallet-ownership challenge before storing the wallet-to-user association.

3.6. The system shall reject wallet linking when the wallet provider is missing, the user rejects the MetaMask request, the signature is invalid, the nonce has expired, or the nonce has already been used.

3.7. The system shall prevent an unauthorized user from linking, replacing, or disconnecting another user's wallet.

3.8. Wallet disconnection shall remove the active association between the user account and wallet without deleting historical transaction records.

3.9. Wallet signing shall occur only through the external wallet provider.

3.10. The application shall not implement its own cryptocurrency private-key custody mechanism.

3.11. The smart contract shall not store user passwords, email addresses, personal profile information, session tokens, or password-reset tokens.

3.12. Smart-contract code shall be tested on a local or test blockchain network before deployment to the target Ethereum environment.

3.13. Smart-contract source code, configuration, and deployment information shall be versioned and reviewed before release.

4. Input Validation and Secure Processing

4.1. The system shall validate all user inputs on the backend even when equivalent frontend validation is present.

4.2. The system shall validate email-address format before account registration, login, and password-reset processing.

4.3. The system shall reject registration when the submitted email address is already associated with an existing account.

4.4. The system shall validate wallet-address format before wallet linking or transaction submission.

4.5. The system shall reject a transfer when the amount is missing, non-numeric, zero, negative, greater than the sender's available balance, or outside an applicable configured limit.

4.6. The system shall prevent users from transferring funds to their own wallet address.

4.7. The system shall validate a registered-user identifier before resolving it to a recipient wallet address.

4.8. The system shall reject unexpected fields, malformed requests, unsupported values, and unauthorized parameter modifications.

4.9. The system shall sanitize or encode user-controlled data before displaying it in the browser.

4.10. Validation errors shall not expose stack traces, database details, server paths, API secrets, or sensitive configuration information.

4.11. The system shall display clear user-facing error messages while storing detailed diagnostic information only in protected logs.

5. Transaction Integrity and Data Consistency

5.1. The system shall generate a unique internal identifier for each transaction request.

5.2. The system shall prevent duplicate transaction submissions using an idempotency mechanism or equivalent server-side control.

5.3. The system shall reject an identical transaction request received while an equivalent request is already being processed.

5.4. The system shall display a transaction-summary screen before submitting a blockchain transaction.

5.5. The transaction-summary screen shall show the receiver and total amount.

5.6. The system shall submit a blockchain transaction only after the authenticated user explicitly confirms the transaction summary.

5.7. If the user cancels the transaction before confirmation, the system shall not submit a blockchain transaction.

5.8. The system shall distinguish between pending, successful, failed, cancelled, and reconciliation-required transaction states.

5.9. The system shall not mark a transaction as successful until a successful blockchain execution result has been received and processed.

5.10. The system shall retain failed transaction records together with the associated failure reason.

5.11. The system shall synchronize MongoDB transaction records with blockchain execution results within 2 seconds after receiving the blockchain response.

5.12. If a blockchain result is available but the database update fails, the system shall preserve enough information to retry reconciliation without submitting a second blockchain transaction.

5.13. If a MongoDB record and blockchain result do not match, the system shall mark the record as requiring reconciliation rather than silently treating the transaction as successful.

5.14. Reconciliation shall compare transaction details and blockchain execution results to verify consistency.

5.15. Temporary application, database, or network failures shall not cause the loss of an already-recorded transaction request.

5.16. The system shall display the blockchain transaction result, including the transaction hash or failure reason, within 2 seconds after receiving the blockchain response.

6. Confidentiality and Data Protection

6.1. The system shall protect sensitive data from unauthorized access during storage, processing, transmission, backup, and recovery.

6.2. The application shall use encrypted transport for browser-to-backend and backend-to-external-service communication.

6.3. MongoDB access shall require authenticated connections and shall follow the principle of least privilege.

6.4. Database credentials, API credentials, email-service credentials, and deployment credentials shall not be hard-coded in source code.

6.5. Application secrets shall be stored outside the source-code repository using environment variables or an equivalent secrets-management mechanism.

6.6. Sensitive secrets shall not be committed to version control, included in screenshots, or written to application logs.

6.7. Backup files containing sensitive data shall be encrypted or otherwise protected from unauthorized access.

6.8. Backup access shall be restricted to authorized personnel or services.

6.9. The system shall not return wallet information, transaction records, or personal data unless the authenticated user is authorized to view that information.

7. Logging, Monitoring, and Auditability

7.1. The system shall log the following security-relevant events:

- successful login
- failed login
- password-reset request
- successful password reset
- wallet connection attempt
- wallet verification success or failure
- wallet disconnection

- transfer attempt
- transfer confirmation
- transfer cancellation
- blockchain execution result
- duplicate submit rejection
- authorisation failure
- administrative action
- transaction-reconciliation attempt
- backup action
- recovery action

7.2. Each audit-log record shall include the event type, timestamp, result, relevant user identifier when available, and relevant wallet or transaction identifier when applicable.

7.3. Logs shall not contain plaintext passwords, password-reset tokens, authentication tokens, private keys, wallet seed phrases, or unnecessary sensitive data.

7.4. Logs shall be protected from unauthorized modification and deletion.

7.5. Access to logs shall be restricted to authorized personnel or services.

7.6. Failed operations shall be logged with sufficient diagnostic detail for investigation.

7.7. Diagnostic details shall not be exposed directly to end users.

7.8. Critical synchronization failures, repeated authorization failures, and repeated duplicate-submit attempts shall be identifiable through logs and monitoring.

8. Backup, Recovery, and Availability

8.1. The system shall back up user-account records, wallet associations, transaction records, and audit logs.

8.2. Backup copies shall be protected from unauthorized access, modification, and deletion.

8.3. The recovery process shall preserve the association between transaction records and blockchain transaction hashes.

8.4. The recovery process shall not create duplicate blockchain transaction submissions.

8.5. Backup restoration shall be tested before the final production release and after material database-schema changes.

8.6. Restored transaction records shall be reconciled with available blockchain results.

8.7. The system shall preserve the final blockchain status when it can be verified after a temporary application or database failure.

8.8. The system shall fail safely when an external dependency is unavailable.

8.9. A dependency failure shall not bypass authentication, authorization, input validation, transaction confirmation, or transaction-integrity controls.

9. Secure Configuration and Deployment

9.1. The production deployment shall use secure default settings.

9.2. Debug mode, verbose stack traces, test accounts, development-only routes, and test credentials shall be disabled in the production environment.

9.3. Production secrets shall not be reused in development or test environments.

9.4. Newly registered users shall not receive administrative privileges by default.

9.5. The system shall not include default administrator passwords or undocumented administrative commands.

9.6. Cross-origin access, exposed API routes, and network access shall be restricted to the minimum required for system operation.

9.7. Security-relevant configuration changes shall be documented and reviewed before deployment.

9.8. The deployed application shall use supported versions of runtimes, frameworks, libraries, and packages.

10. Secure Development and Source-Code Protection

10.1. Source code, smart-contract code, configuration files, and deployment scripts shall be stored in a version-control repository.

10.2. Repository access shall follow the principle of least privilege.

10.3. Repository accounts with write access shall use multi-factor authentication when supported.

10.4. Changes to security-sensitive code shall be reviewed before merging.

10.5. Security-sensitive code includes authentication, authorization, wallet verification, transaction submission, smart-contract integration, database synchronization, logging, backup, and recovery logic.

10.6. Source-code changes shall be traceable to an identified contributor and commit.

10.7. Secrets shall not be stored in the repository.

10.8. Development dependencies, build tools, runtime versions, and package versions shall be documented.

10.9. Supported and maintained versions of tools and dependencies shall be used where feasible.

10.10. Security-related development artifacts shall be retained, including risk decisions, review notes, test results, dependency inventories, approved exceptions, and release records.

11. **Third-Party Component and Software-Supply-Chain Security**

11.1. The project shall maintain an inventory of third-party frameworks, libraries, packages, smart-contract dependencies, wallet-provider dependencies, and external services.

11.2. The dependency inventory shall record the component name, version, source, purpose, and update status.

11.3. Third-party components shall be reviewed for publicly known vulnerabilities before a production release.

11.4. Unsupported, abandoned, or end-of-life dependencies shall be replaced or mitigated before release where feasible.

11.5. The dependency inventory shall be updated whenever a component is added, removed, or upgraded.

11.6. The project shall retain a Software Bill of Materials or equivalent dependency record for each release.

11.7. A third-party component with an unresolved high-risk vulnerability shall not be included in a production release unless a documented exception, justification, mitigation plan, and approval exist.

12. Security Verification and Release Criteria

12.1. The project shall maintain a threat model or equivalent security-risk model.

12.2. Security-sensitive code shall undergo code review or automated analysis before production release.

12.3. Security testing shall include positive, negative, boundary, and authorization test cases.

12.4. Security testing shall cover authentication, wallet verification, transaction processing, authorization controls, blockchain integration, database synchronization, backup restoration, and recovery scenarios.

12.5. Discovered vulnerabilities shall be recorded with their severity, affected component, remediation action, responsible person, and verification status.

12.6. A production release shall not contain unresolved critical vulnerabilities.

12.7. An unresolved high-risk vulnerability shall require a documented exception, justification, mitigation plan, and approval before release.

12.8. Regression tests shall be added for corrected security defects where feasible.

12.9. Each release record shall include the application version, source-code commit identifier, dependency inventory, configuration version, test results, known limitations, and approved exceptions.

12.10. Release files and supporting artifacts shall be archived in a protected location.

13. Vulnerability Monitoring and Response

13.1. The project shall monitor supported third-party components and dependencies for newly reported vulnerabilities.

13.2. Credible vulnerability reports shall be recorded, investigated, prioritized, and tracked until closure.

13.3. Each confirmed vulnerability shall be evaluated based on its exploitability, potential impact, affected users or components, and available mitigation.

13.4. Vulnerabilities shall be prioritized using a risk-based approach.

13.5. When a permanent fix is not immediately available, the project shall document a temporary mitigation or workaround where feasible.

13.6. Corrected vulnerabilities shall be retested before release.

13.7. The project shall analyse the root cause of significant vulnerabilities.

13.8. When a vulnerability reveals a recurring weakness, similar code paths, dependencies, and configurations shall be reviewed for related issues.

13.9. Lessons learned from significant vulnerabilities shall be used to improve future development and testing activities.

13.10. The project documentation shall identify a method for reporting suspected security vulnerabilities to the project maintainer.

5.2. Use Case Diagram

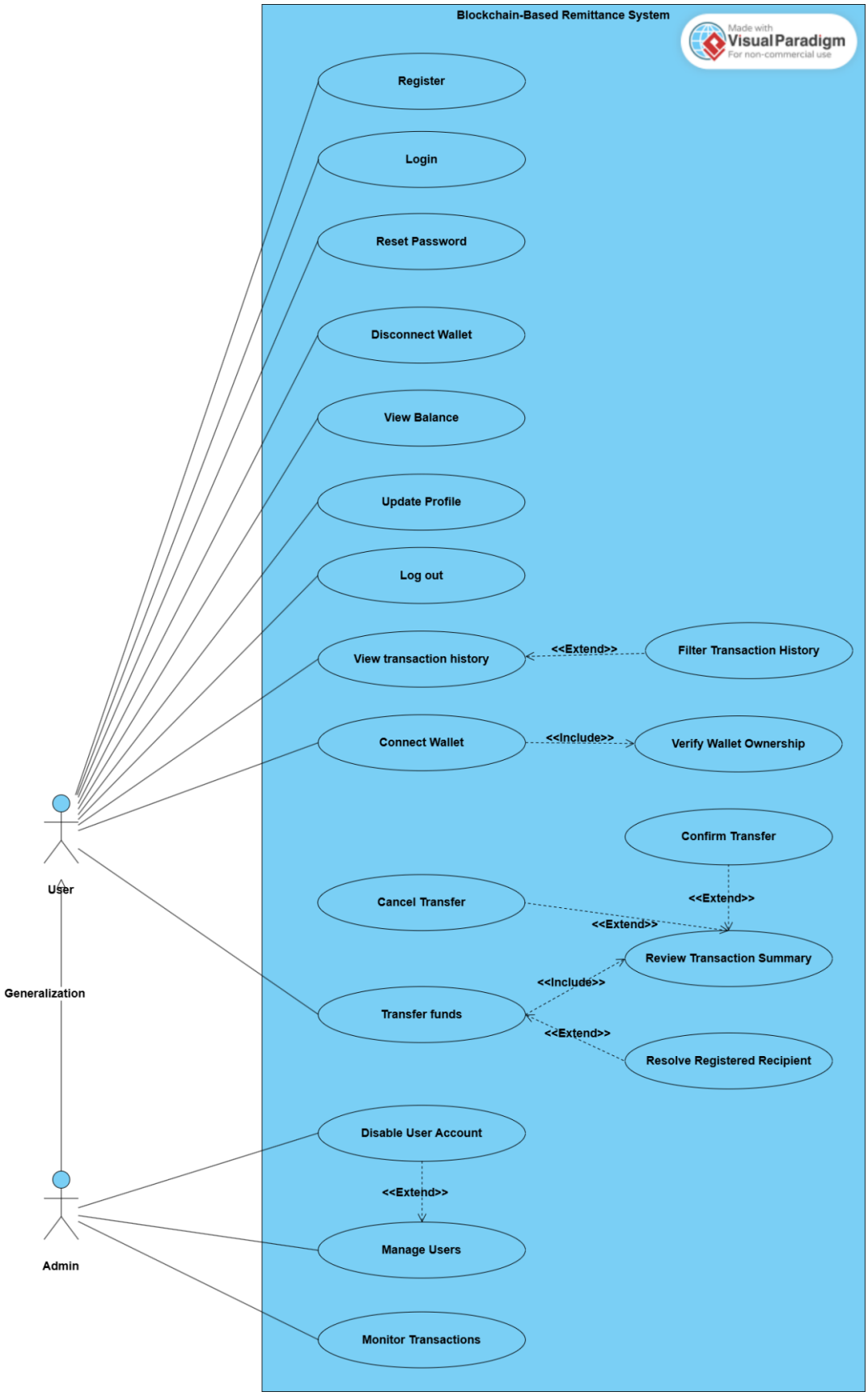


Figure 6: Use Case Diagram

5.3. Use Case Description

Name	Connect a Wallet
Description	Connects a MetaMask wallet to the authenticated user after verifying wallet ownership.
Actor(s)	User
Preconditions	The user must be logged in to the system.
Postconditions	The verified wallet address is linked to the user account.
Flow	1. User clicks Connect Wallet. 2. System checks whether the MetaMask wallet provider is available. 3. System requests access to the wallet through MetaMask. 4. User approves the wallet-connection request. 5. System generates a unique one-time ownership-verification challenge. 6. User signs the verification challenge through MetaMask. 7. System verifies the signature, nonce, expiration time, and one-time-use status. 8. System links the verified wallet address to the user account. 9. System displays a wallet connected successfully message.
Alternative Flow	1. MetaMask is not available: system displays a wallet provider not found message. 2. User rejects the wallet connection request: system does not link the wallet and displays a connection-denied message. 3. User rejects the ownership signature request: system does not link the wallet and displays a verification failure message. 4. Signature is invalid, expired, or already used: system rejects the verification attempt.

Name	Reset Password
Description	Allows a user to reset a forgotten password through a secure recovery process.
Actor(s)	User
Preconditions	The user is not logged in and has access to the registered email address.
Postconditions	The password is updated securely, the reset link becomes invalid, and active sessions are invalidated.
Flow	1. User opens the password-reset page. 2. User enters the registered email address. 3. System validates the email-address format. 4. System generates a secure one-time password-reset token. 5. System stores a protected representation of the token with a 15-minute expiration time. 6. System sends a password reset link to the registered email address. 7. User opens the link and enters a new password. 8. System verifies that the link is valid, unused, and not expired. 9. System validates the new password. 10. System securely stores the new password hash. 11. System invalidates the reset link and active sessions. 12. System displays a password reset success message.
Alternative Flow	1. Email format is invalid: system displays an inline validation message. 2. Email field is empty: system asks the user to complete the required field. 3. Reset link has expired: system rejects the request and displays a link expired message. 4. Reset link was already used: system rejects the request and displays an invalid link message. 5. New password does not satisfy the password rules: system displays a password validation message.

Name	Transfer funds
Description	Allows an authenticated user to send ETH to a valid wallet address or registered user.
Actor(s)	User
Preconditions	The user must be logged in, have a verified connected wallet, and have sufficient ETH balance.
Postconditions	The transaction is submitted once, recorded in the database, and displayed in the transaction history with its status.
Flow	1. User opens the transfer funds page. 2. User enters a receiver wallet address or registered-user identifier. 3. User enters the ETH amount. 4. System validates the receiver and amount. 5. If a registered user identifier is entered, system resolves it to the linked wallet address. 6. System verifies that the receiver is valid and that the sender has sufficient balance. 7. System displays a transaction-summary screen showing the receiver and total amount. 8. User reviews the summary and clicks Confirm Transfer. 9. System creates a unique transaction identifier and prevents duplicate submission. 10. System stores the transaction as pending. 11. Blockchain executes the transaction. 12. System receives the transaction hash and execution status. 13. System updates the MongoDB transaction record. 14. System displays a result modal within 2 seconds after receiving the blockchain response.
Alternative Flow	1. Wallet address is invalid: system displays a red inline wallet address error. 2. Registered-user identifier is not found: the system displays a recipient not found message. 3. Amount is missing, zero, negative, or greater than the available balance: system displays an amount-validation message. 4. User clicks Cancel Transfer before confirmation: system cancels the operation and sends no blockchain transaction. 5. Duplicate request is detected: system rejects the repeated submission. 6. Blockchain execution fails: the system stores the failed status and displays Transaction Failed with the specific reason. 7. Blockchain result is received but the database update fails: the system marks the transaction as requiring reconciliation and does not submit a second blockchain transaction.

Name	View Transaction History
Description	Displays the authenticated user's previous transactions and allows optional filtering.
Actor(s)	User
Preconditions	The user must be logged in to the system.
Postconditions	The user's transaction history is displayed.
Flow	1. User opens the transaction history page. 2. System verifies that the user is authenticated. 3. System loads the transactions related to the user. 4. System displays the sender wallet, receiver wallet, timestamp, amount, status, and transaction hash when available. 5. User may optionally filter transactions by date range, transaction type, or status. 6. System applies the selected filters. 7. System displays the matching transaction records.
Alternative Flow	1. No matching transactions are found: the system displays "No transactions found". 2. User is not logged in: system redirects the user to the login page. 3. Transaction records cannot be retrieved: system displays a temporary error message and allows the user to retry.

Name	Update Profile
Description	Allows the authenticated user to modify permitted profile information.
Actor(s)	User
Preconditions	The user must be logged in to the system.
Postconditions	Valid profile changes are saved successfully.
Flow	1. User opens the profile page. 2. User clicks Edit Profile. 3. User modifies the permitted fields. 4. System validates the entered data. 5. User clicks Save Changes. 6. System stores the valid changes. 7. System displays a profile updated successfully message.
Alternative Flow	1. Entered data is invalid: system displays inline validation messages below the affected fields. 2. A required field is empty: system asks the user to complete the required field. 3. User attempts to modify a restricted field: system rejects the request.

Name	Disable User Account
Description	Allows an authenticated admin to disable an active user account.
Actor(s)	Admin
Preconditions	The admin must be logged in and have the admin role.
Postconditions	The selected account is disabled, active sessions are invalidated, and future login attempts are blocked.
Flow	1. Admin opens the user-management page. 2. Admin selects an active user account. 3. Admin clicks Disable User Account. 4. System asks the admin to confirm the action. 5. Admin confirms the action. 6. System marks the selected account as disabled. 7. System invalidates active sessions associated with the disabled account. 8. System records the administrative action. 9. System displays an account disabled successfully message.
Alternative Flow	1. Admin cancels the action: account remains active. 2. Selected account is already disabled: the system informs the administrator that no change is required. 3. Selected account cannot be found: the system displays a user not found message. 4. Unauthorized request is submitted: the system rejects the action.

Name	Manage user
Description	Allows an authenticated admin to view registered user accounts and their account status.
Actor(s)	Admin
Preconditions	The admin must be logged in and have the admin role.
Postconditions	The admin can view registered user accounts and select an account for an approved management action.
Flow	1. Admin opens the admin dashboard. 2. System verifies the admin's authenticated session and role. 3. Admin opens the user-management page. 4. System displays the registered user accounts and their status. 5. Admin selects a user account. 6. System displays the permitted account management actions. 7. Admin may choose to disable an active account.
Alternative Flow	1. A regular user attempts to access the page: system denies access. 2. Admin is not logged in: system redirects the administrator to the login page. 3. Selected account cannot be found: the system displays a user not found message.

Name	Monitor transactions
Description	Allows an authenticated admin to view and monitor system transaction records.
Actor(s)	Admin
Preconditions	The admin must be logged in and have the admin role.
Postconditions	The admin can view the available transaction records and their status.
Flow	1. Admin opens the admin dashboard. 2. System verifies the admin's authenticated session and role. 3. Admin opens the transaction monitoring page. 4. System retrieves the transaction records. 5. System displays the sender wallet, receiver wallet, amount, timestamp, transaction hash when available, and transaction status. 6. System indicates whether each transaction is pending, successful, failed, cancelled, or requires reconciliation.
Alternative Flow	1. No transaction records exist: the system displays No transactions found. 2. A regular user attempts to access the page: system denies access. 3. Admin is not logged in: system redirects the admin to the login page. 4. Transaction records cannot be retrieved: system displays a temporary error message and allows the admin to retry.

5.4. Class Diagram

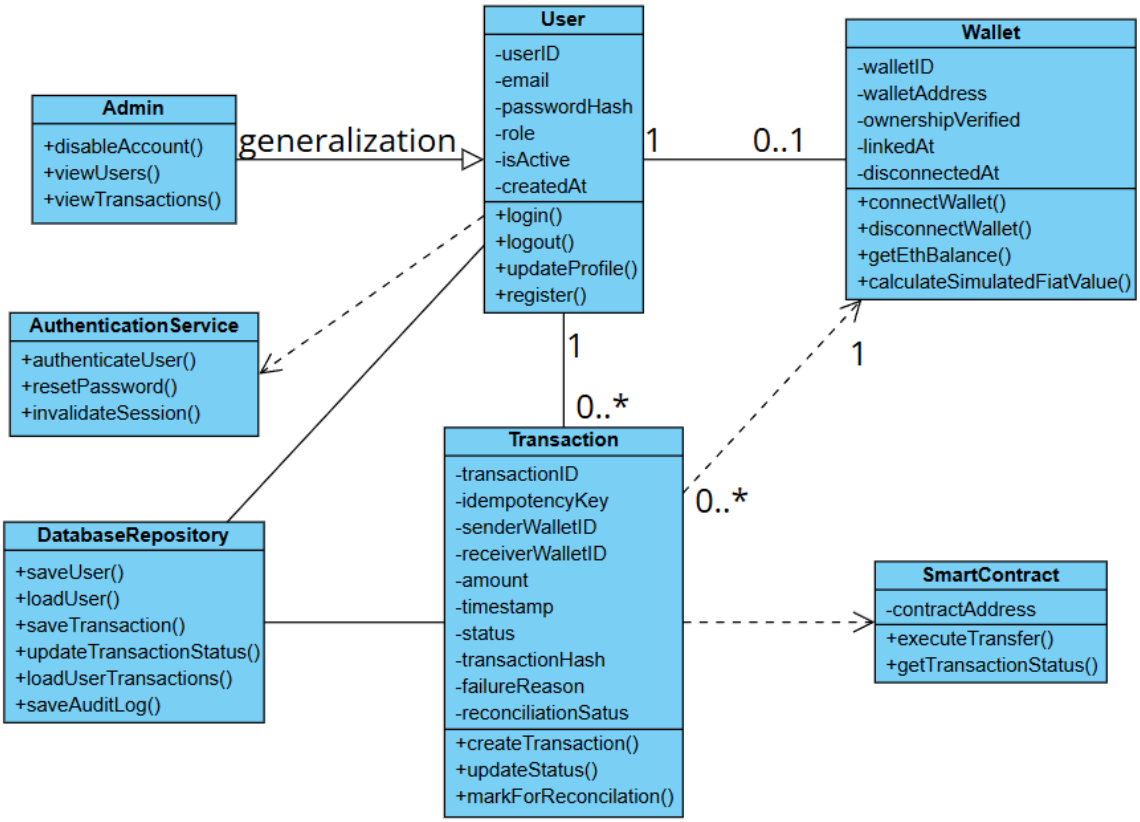


Figure 7: Class Diagram

5.5. Sequence Diagram

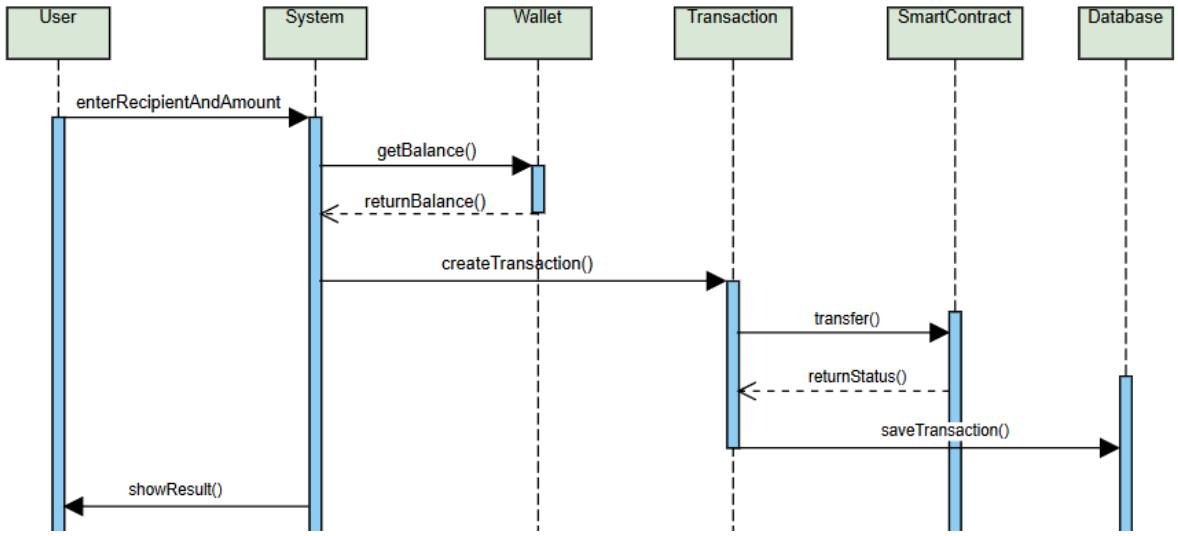


Figure 8: Sequence Diagram of Transferring Funds

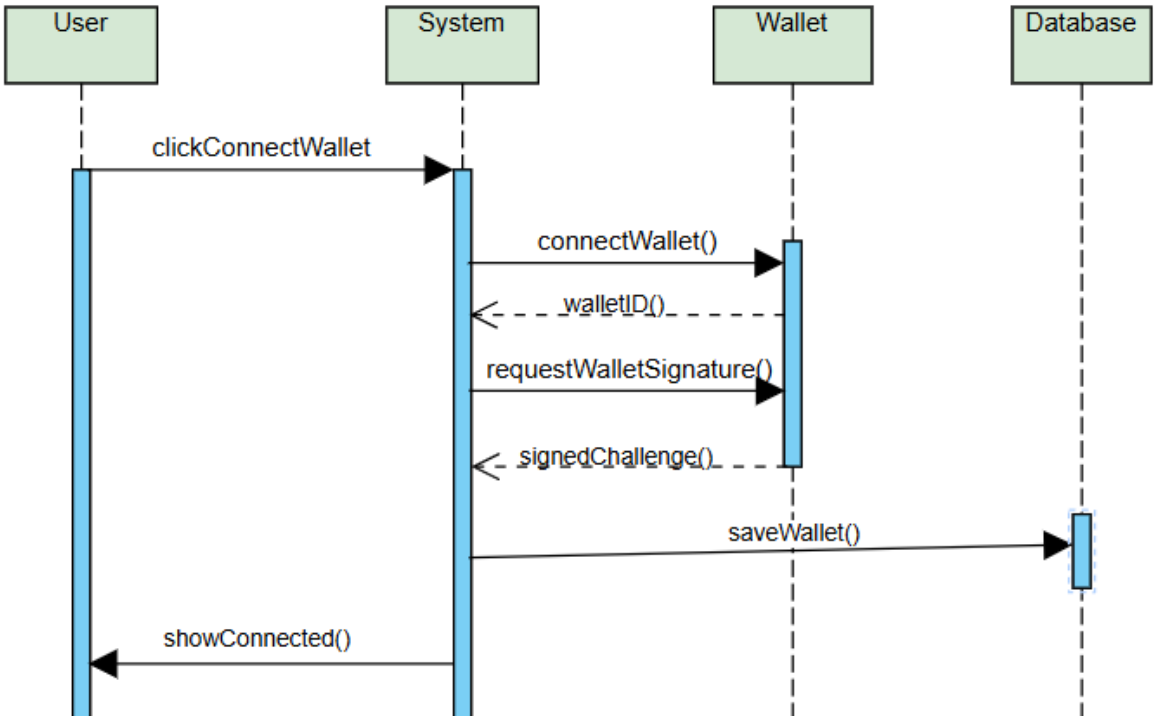


Figure 9: Sequence Diagram of Connect to Wallet

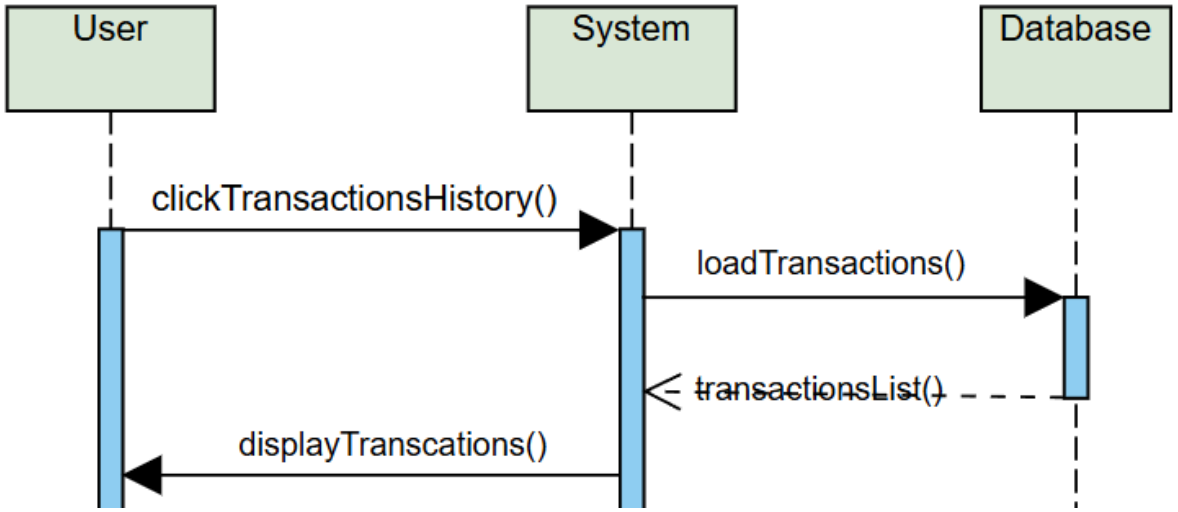


Figure 10: Sequence Diagram of Displaying Transactions History

5.6. Activity Diagram

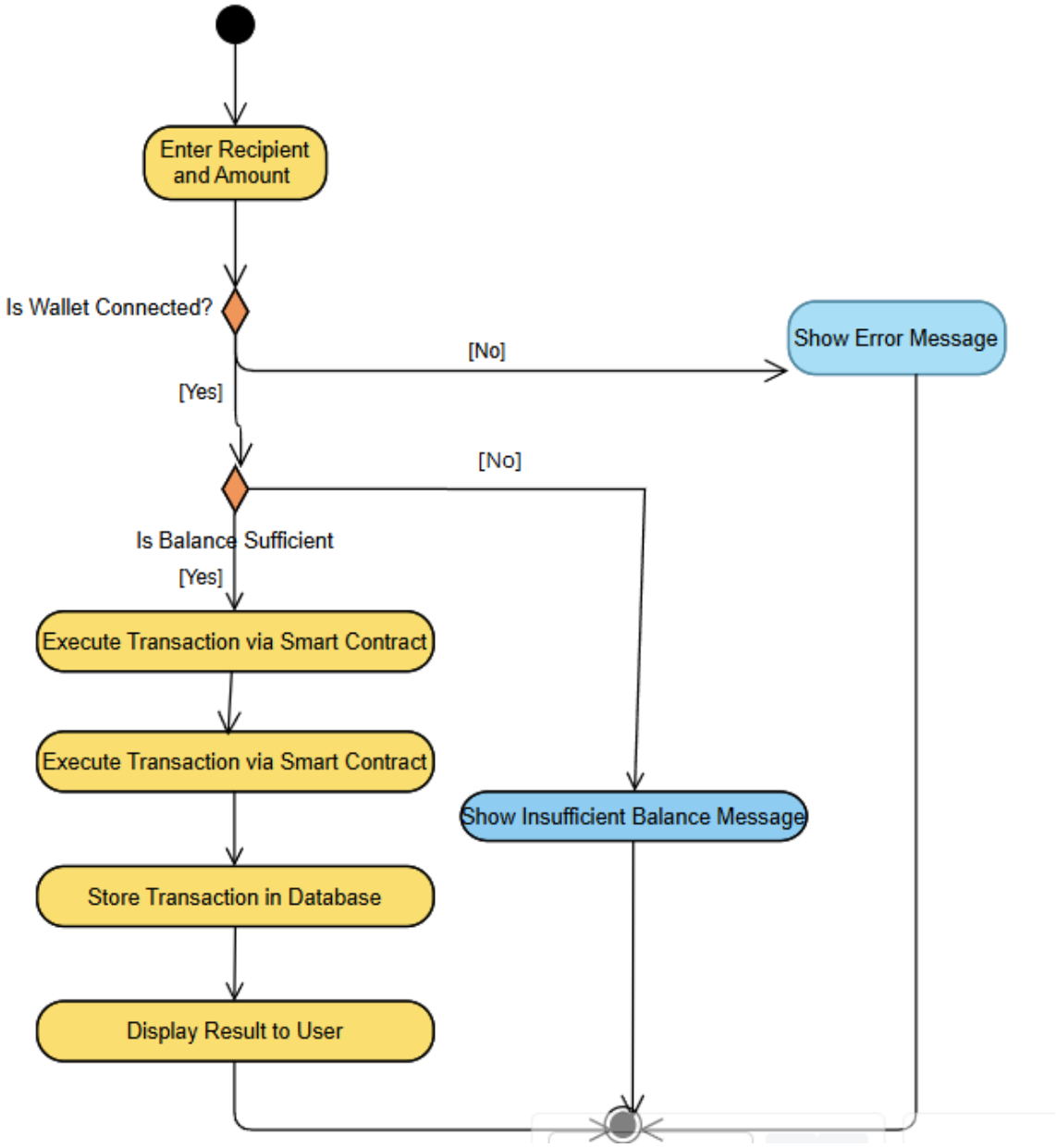


Figure 11: Activity Diagram of Transfer Funds

5.7. ER Diagram

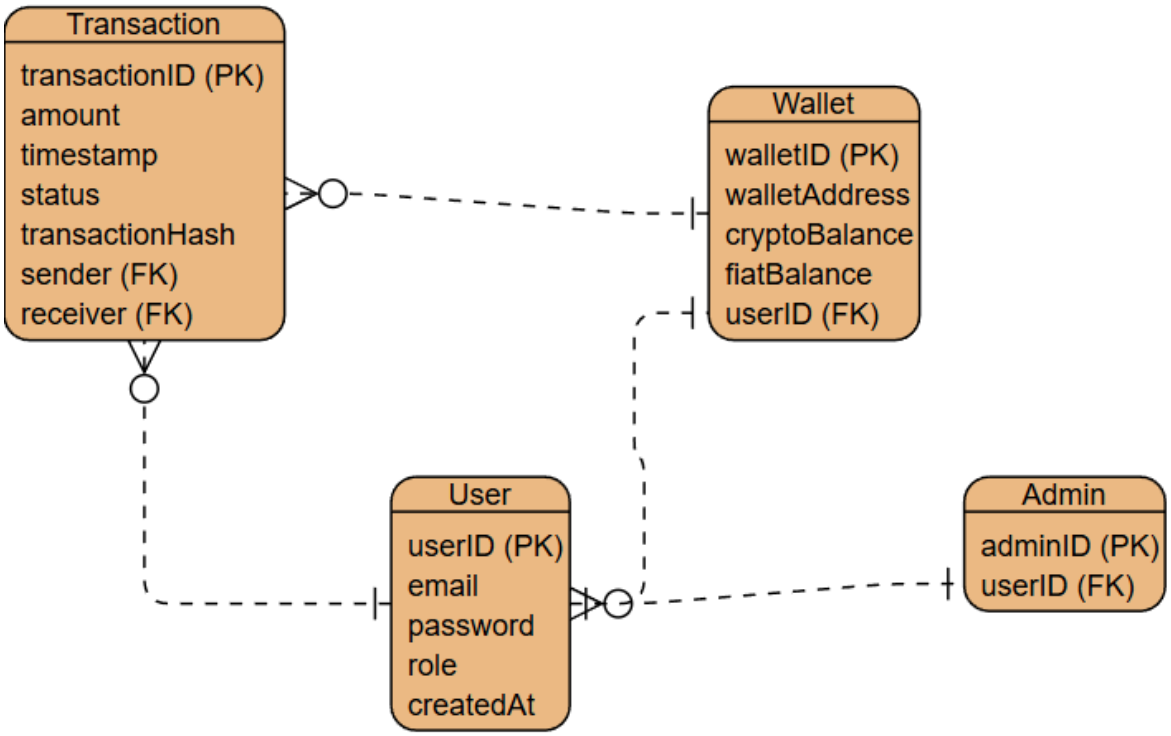


Figure 12: ER Diagram

5.8. Data Flow Diagram

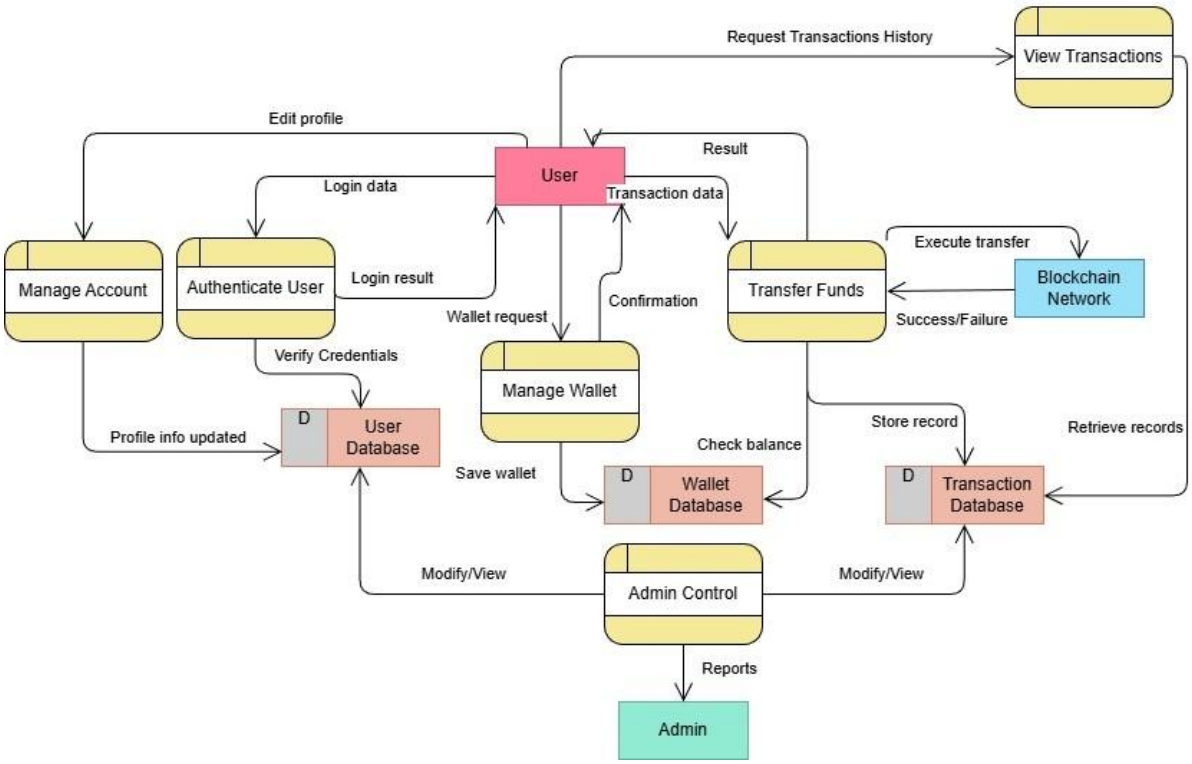


Figure 13: System Data Flow Diagram

6. Testing Process

6.1. Registration

No	Type	Description	Example Input	Expected Output
1	NC	Register with valid unique email and strong password	Email: ahmad@test.com Password: Test@123	Account is created successfully and user receives success message.
2	EC	Register using an email that already exists	Email: [registered email]	System rejects registration and shows an error that email is already used.
3	EC	Register with invalid email format	Email: ahmadtest.com	Red inline error appears below email field.
4	EC	Register with password less than 8 characters	Password: T@12	System rejects password and shows password rule error.
5	EC	Register with password missing uppercase letter	Password: test@123	System rejects password and shows strong-password validation message.
6	EC	Register with password missing special character	Password: Test1234	System rejects password and shows strong-password validation message.
7	EC	Submit registration form with empty required fields	Email: [empty] Password: [empty]	System asks user to complete required fields using inline validation messages.

6.2. Log in

No	Type	Description	Example Input	Expected Output
1	NC	Log in with registered email and correct password	Email: ahmad@test.com Password: Test@123	User is logged in successfully and redirected to dashboard.
2	EC	Log in with correct email and wrong password	Email: ahmad@test.com Password: Wrong@123	System rejects login and shows generic invalid credentials message.
3	EC	Log in with unregistered email	Email: unknown@test.com	System rejects login and shows generic invalid credentials message.
4	EC	Submit login form with empty fields	Email: [empty] Password: [empty]	System asks user to complete required fields.
5	EC	Log in with invalid email format	Email: ahmad.com	Red inline error appears below email field.
6	NC	Check that authenticated user can open dashboard	User logs in successfully	Dashboard opens and protected functions become available.

7	EC	Disabled account attempts to log in	Email: [email of a disabled user] Password: [correct password]	System blocks login and shows account disabled message.

6.3. Connect Wallet and View Balance

No	Type	Description	Example Input	Expected Output
1	NC	Check successful wallet connection	User clicks Connect Wallet and approves MetaMask request	Wallet address is linked to the logged-in user account.
2	EC	Check wallet connection when user rejects request	User clicks Reject in MetaMask	Wallet is not connected and system shows a connection denied message.
3	EC	Check wallet connection when wallet provider is missing	Browser without MetaMask installed	System shows wallet provider not found message.
4	EC	Check wallet ownership verification failure	User connects wallet but rejects ownership signature	System does not link wallet and shows verification failure message.
5	EC	Check wallet balance display in ETH	Connected wallet balance: 1.25 ETH	Dashboard displays the current ETH wallet balance.
6	NC	Check simulated fiat value display	Balance: 1 ETH Fixed rate: 3000 USD	Dashboard displays simulated fiat value as 3000 USD.
7	EC	Check balance view when no wallet is connected	Logged-in user with no connected wallet	System shows a message that no wallet is connected.

6.4. Transfer Funds

No	Type	Description	Example Input	Expected Output
1	NC	Check successful transfer to valid wallet address	Receiver: 0x937… Amount: 0.1 ETH	Confirmation screen appears, user confirms, and transaction is executed.
2	NC	Check transfer to registered user using email	Receiver: mohammad@test.com Amount: 0.05 ETH	System resolves receiver wallet and executes transfer after confirmation.
3	EC	Check transfer with invalid wallet address	Receiver: 0x12345 Amount: 0.1 ETH	System rejects receiver and shows inline wallet-address error.
4	EC	Check transfer with amount greater than balance	Balance: 0.2 ETH Amount: 1 ETH	System prevents transfer and shows insufficient balance message.

5	EC	Check transfer with missing amount	Receiver entered, amount empty	System shows required amount validation message.
6	EC	Check transfer with zero or negative amount	Amount: 0 ETH or -1 ETH	System rejects amount and shows invalid amount message.
7	NC	Check cancel transaction before confirmation	Valid receiver and amount, user clicks Cancel	Transaction is cancelled and no blockchain transaction is sent.

6.5. View and Filter Transaction History

No	Type	Description	Example Input	Expected Output
1	NC	Check full transaction history display	User opens Transaction History page	System displays all transactions related to the user.
2	NC	Check transaction details completeness	Existing transaction record	Record shows sender wallet, receiver wallet, timestamp, amount, status, and transaction hash.
3	NC	Check filter by date range	Date range: 2026-04-20 to 2026-04-27	System displays only transactions within the selected date range.
4	NC	Check filter by transaction type	Type: Sent	System displays only sent transactions.
5	NC	Check filter by transaction status	Status: Failed	System displays only failed transactions.
6	EC	Check filter with no matching transactions	Future date range with no transactions	System shows “No transactions found.”
7	EC	Check transaction history access without login	Open transaction history URL directly	System redirects the user to the login page.

6.6. Reset Password

No	Type	Description	Example Input	Expected Output
1	NC	Request a password-reset link using a registered email address	Email: ahmad@test.com	System accepts the request, sends a password-reset link to the registered email address, and displays a confirmation message.
2	EC	Request a password-reset link using an invalid email format	Email: ahmadtest.com	System rejects the request and displays a red inline validation message below the email field.
3	EC	Submit the password-reset request form with an empty email field	Email: [empty]	System asks the user to complete the required field using an inline validation message.

4	EC	Attempt to use an expired password-reset link	Reset link generated more than 15 minutes ago	System rejects the reset attempt and displays a message indicating that the password-reset link has expired.
5	EC	Attempt to reuse a password-reset link after a successful password reset	Previously used reset link	System rejects the reset attempt and displays a message indicating that the password-reset link is invalid or has already been used.
6	EC	Enter a new password that does not satisfy the password rules	New Password: test1234	System rejects the new password and displays a message stating that the password must contain at least 8 characters, including an uppercase letter, a number, and a special character.
7	NC	Reset the password using a valid unexpired one-time link and a strong password	Valid reset linkNew Password: NewTest@123	System updates the password successfully, invalidates the reset link, invalidates active sessions, and displays a success message.

6.7. Admin

No	Type	Description	Example Input	Expected Output
1	NC	Check that an authenticated admin can open the admin dashboard	Admin logs in successfully and opens the admin dashboard	System grants access and displays the administrative functions.
2	EC	Check that a regular user cannot access the admin dashboard	Authenticated regular user opens the admin dashboard URL directly	System denies access and displays an unauthorized-access message or redirects the user to an allowed page.
3	EC	Check admin dashboard access without login	Unauthenticated user opens the admin-dashboard URL directly	System redirects the user to the login page.
4	NC	View registered user accounts	Admin opens the user-management page	System displays the registered user accounts together with their account status.
5	NC	Disable an active user account	Admin selects an active account and clicks Disable Account	System updates the account status to disabled, records the administrative action in the audit log, and displays a success message.

6	EC	Check that a disabled user cannot log in after the admin disables the account	Email: [email of disabled user] Password: [correct password]	System blocks login and displays an account disabled message.
7	NC	Monitor transaction records and their statuses	Admin opens the transaction monitoring page	System displays transaction records with sender wallet, receiver wallet, amount, timestamp, transaction hash when available, and status such as pending, successful, failed, or reconciliation required.

7. Project Management

7.1. Major Project Phases

No.	Phase Name
1	Planning and SRS alignment
2	Design preparation
3	Environment and project setup
4	Core implementation
5	Security and reliability hardening
6	Testing and quality control
7	Documentation and final delivery

7.2. Breakdown Structure

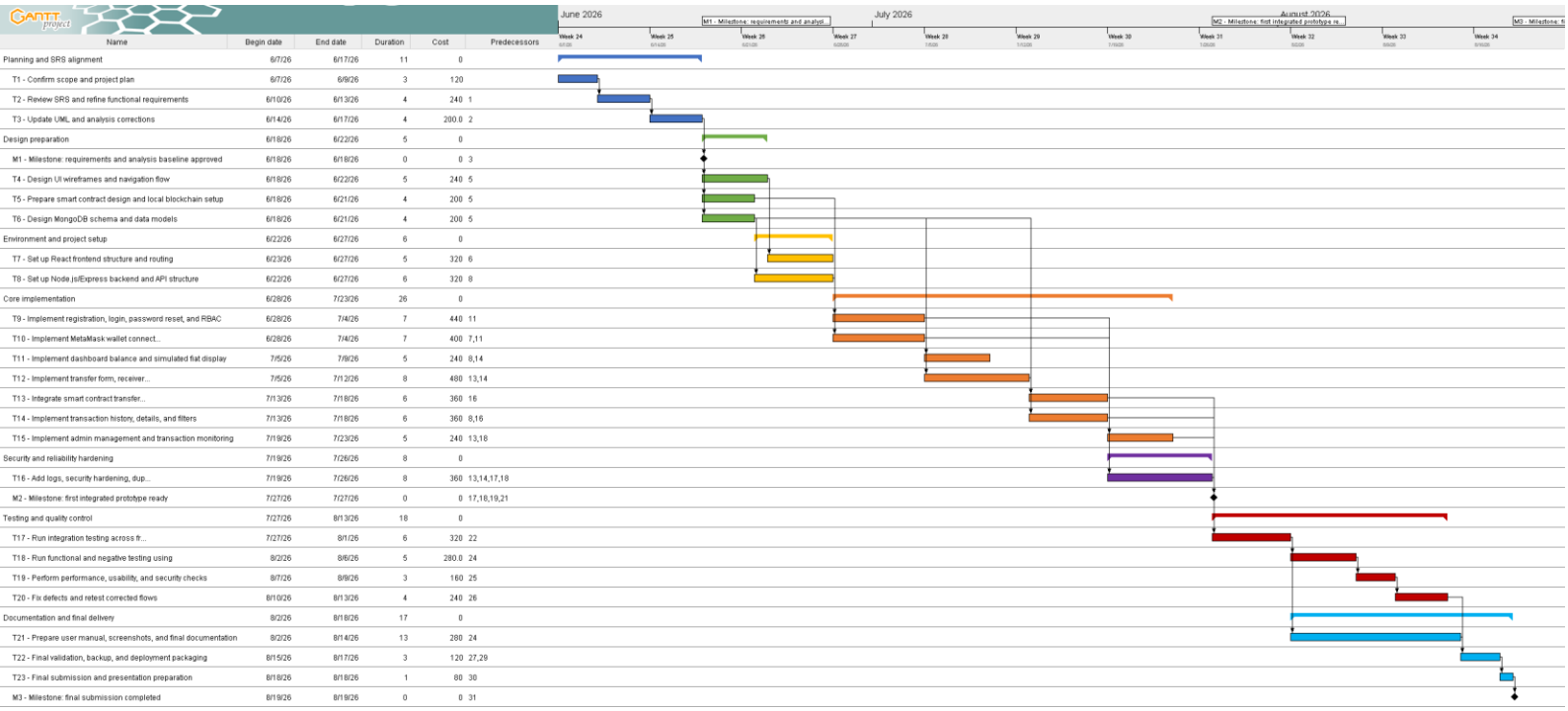
Level	Task ID	Task Name
1	1.0	Blockchain-Based Remittance Web System
2	1.1	Planning and SRS alignment
3	1.1.1	Confirm scope and project plan
3	1.1.2	Review SRS and refine functional requirements
3	1.1.3	Update UML and Analysis corrections
2	1.2	Design preparation
3	1.2.1	Design UI wireframes and navigation flow
3	1.2.2	Prepare smart contract design and local blockchain setup
3	1.2.3	Design MongoDB schema and data models
2	1.3	Environment and project setup
3	1.3.1	Set up React frontend structure and routing
3	1.3.2	Set up Node.js/Express backend and API structure
2	1.4	Core implementation
3	1.4.1	Implement registration, login, password reset and RBAC
3	1.4.2	Implement dashboard balance and simulated fiat display
3	1.4.3	Implement transfer form, receiver validation, confirmation and cancel flow
3	1.4.4	Integrate smart contract transfer execution and transaction status handling
3	1.4.5	Implement transaction history, details and filters
3	1.4.6	Implement admin management and transaction monitoring
3	1.4.7	Security and reliability hardening
2	1.5	Security and Reliability Hardening
3	1.5.1	Add logs, security hardening, duplicates prevention and backups
2	1.6	Testing and quality control

3	1.6.1	Run integration testing across frontend, backend, wallet, smart contract and database
3	1.6.2	Run functional and negative testing
3	1.6.3	Perform performance, usability and security checks
3	1.6.4	Fix defects and retest corrected flows
2	1.7	Documentation and final delivery
3	1.7.1	Prepare user manual, screenshots and final documentation
3	1.7.2	Final validation, backup and deployment packaging
3	1.7.3	Final submission

7.3. Dependency Table

Task ID	Task Name	Predecessor(s)
T1	Confirm scope and project plan	None
T2	Review SRS and refine functional requirements	T1
T3	Update UML and analysis corrections	T2
M1	**Milestone: requirements and analysis baseline approved**	**T3**
T4	Design UI wireframes and navigation flow	M1
T5	Prepare smart contract design and local blockchain setup	M1
T6	Design MongoDB schema and data models	M1
T7	Set up React frontend structure and routing	T4
T8	Set up Node.js/Express backend and API structure	T6
T9	Implement registration, login, password reset, and RBAC	T8
T10	Implement MetaMask wallet connection and ownership verification	T5, T8
T11	Implement dashboard balance and simulated fiat display	T10, T6
T12	Implement transfer form, receiver validation, confirmation, and cancel flow	T9, T10
T13	Integrate smart contract transfer execution and transaction status handling	T12
T14	Implement transaction history, details, and filters	T6, T12
T15	Implement admin management and transaction monitoring	T9, T14
T16	Add logs, security hardening, duplicate-submit prevention, and backups	T9, T10, T13, T14
M2	**Milestone: first integrated prototype ready**	**T13, T14, T15, T16**
T17	Run integration testing across frontend, backend, wallet, smart contract, and database	M2
T18	Run functional and negative testing	T17
T19	Perform performance, usability, and security checks	T18
T20	Fix defects and retest corrected flows	T19
T21	Prepare user manual, screenshots, and final documentation	T17
T22	Final validation, backup, and deployment packaging	T20, T21
T23	Final submission and presentation preparation	T22

7.4. Gantt Chart Diagram



Name	Begin date	End date	Duration	Cost	Predecessors
Planning and SRS alignment	6/7/26	6/17/26	11	0	
T1 - Confirm scope and project plan	6/7/26	6/9/26	3	120	
T2 - Review SRS and refine functional requirements	6/10/26	6/13/26	4	240	1
T3 - Update UML and analysis corrections	6/14/26	6/17/26	4	200.0	2
Design preparation	6/18/26	6/22/26	5	0	
M1 - Milestone: requirements and analysis baseline approved	6/18/26	6/18/26	0	0	3
T4 - Design UI wireframes and navigation flow	6/18/26	6/22/26	5	240	5
T5 - Prepare smart contract design and local blockchain setup	6/18/26	6/21/26	4	200	5
T6 - Design MongoDB schema and data models	6/18/26	6/21/26	4	200	5
Environment and project setup	6/22/26	6/27/26	6	0	
T7 - Set up React frontend structure and routing	6/23/26	6/27/26	5	320	6
T8 - Set up Node.js/Express backend and API structure	6/22/26	6/27/26	6	320	8
Core implementation	6/28/26	7/23/26	26	0	
T9 - Implement registration, login, password reset, and RBAC	6/28/26	7/4/26	7	440	11
T10 - Implement MetaMask wallet connect...	6/28/26	7/4/26	7	400	7,11
T11 - Implement dashboard balance and simulated fiat display	7/5/26	7/8/26	5	240	8,14
T12 - Implement transfer form, receiver...	7/5/26	7/13/26	8	480	13,14
T13 - Integrate smart contract transfer...	7/13/26	7/18/26	6	360	16
T14 - Implement transaction history, details, and filters	7/13/26	7/18/26	6	360	8,16
T15 - Implement admin management and transaction monitoring	7/19/26	7/23/26	5	240	13,18
Security and reliability hardening	7/19/26	7/26/26	8	0	
T16 - Add logs, security hardening, dup...	7/19/26	7/26/26	8	360	13,14,17,18
M2 - Milestone: first integrated prototype ready	7/27/26	7/27/26	0	0	17,18,19,21
Testing and quality control	7/27/26	8/13/26	18	0	
T17 - Run integration testing across fr...	7/27/26	8/1/26	6	320	22
T18 - Run functional and negative testing using	8/2/26	8/6/26	5	280.0	24
T19 - Perform performance, usability, and security checks	8/7/26	8/9/26	3	160	25
T20 - Fix defects and retest corrected flows	8/10/26	8/13/26	4	240	26
Documentation and final delivery	8/2/26	8/18/26	17	0	
T21 - Prepare user manual, screenshots, and final documentation	8/2/26	8/14/26	13	280	24
T22 - Final validation, backup, and deployment packaging	8/15/26	8/17/26	3	120	27,29
T23 - Final submission and presentation preparation	8/18/26	8/18/26	1	80	30
M3 - Milestone: final submission completed	8/19/26	8/19/26	0	0	31

Figure 14:Gantt Chart Diagram

7.5. Cost Estimation

Task ID	Task / Resource	Estimated Cost
T1	Confirm scope and project plan	0 USD
T2	Review SRS and refine functional requirements	0 USD
T3	Update UML and analysis corrections	0 USD
T4	Design UI wireframes and navigation flow	0 USD
T5	Prepare smart contract design and local blockchain setup	0 USD
T6	Design MongoDB schema and data models	0 USD
T7	Set up React frontend structure and routing	0 USD
T8	Set up Node.js/Express backend and API structure	0 USD
T9	Implement registration, login, password reset, and RBAC	0 USD
T10	Implement MetaMask wallet connection and ownership verification	0 USD
T11	Implement dashboard balance and simulated fiat display	0 USD

T12	Implement transfer form, receiver validation, confirmation, and cancel flow	0 USD
T13	Integrate smart contract transfer execution and transaction status handling	0 USD
T14	Implement transaction history, details, and filters	0 USD
T15	Implement admin management and transaction monitoring	0 USD
T16	Add logs, security hardening, duplicate-submit prevention, and backups	0 USD
T17	Run integration testing across frontend, backend, wallet, smart contract, and database	0 USD
T18	Run functional and negative testing	0 USD
T19	Perform performance, usability, and security checks	0 USD
T20	Fix defects and retest corrected flows	0 USD
T21	Prepare user manual, screenshots, and final documentation	0 USD
T22	Final validation, backup, and deployment packaging	0 USD
T23	Final submission and presentation preparation	0 USD
D1	Domain/hosting or deployment environment	0 USD
D2	Printing/binding/final submission material	0 USD
D3	Internet/electricity/testing support allowance	49 USD/month
D4	Figma, GitHub, VS Code, Postman, MetaMask, MongoDB community and other tools	19-199 USD/month
D5	Testnet blockchain gas for academic testing	0 USD
Total		68-248 USD/month

7.6. Risk Management

7.6.1. Risk Identification and Classification Table

Risk ID	Risk Description	Risk Type
R-01	Requirements for wallet flow, confirmation or admin functions may change after implementation starts, causing rework.	Requirement Risk
R-02	MetaMask connection or wallet ownership verification may fail when the provider is missing, rejected, or handled incorrectly.	Technical Risk
R-03	Smart contract logic or deployment errors may prevent transfers or produce unreliable transaction status.	Technical Risk

R-04	Frontend, backend, smart contract, and MongoDB may not synchronize transaction hash, sender, receiver, amount and status correctly.	Integration Risk
R-05	Weak authentication, RBAC or input validation may allow unauthorized access, invalid transfers, or admin misuse.	Security Risk
R-06	MongoDB transaction history may not match blockchain results because of failed updates, duplicate submits or incomplete reconciliation.	Data Risk
R-07	Testing may miss negative cases such as invalid wallet address, insufficient balance, rejected signature, duplicate submit and empty fields.	Quality Risk
R-08	Implementation may take longer than planned because blockchain, MetaMask and smart contract debugging require extra time.	Schedule Risk

7.6.2. Risk Matrix Table

Risk ID	Risk Description	Probability	Impact	Risk Level
R-01	Requirements for wallet flow, confirmation or admin functions may change after implementation starts, causing rework.	Medium	High	High
R-02	MetaMask connection or wallet ownership verification may fail when the provider is missing, rejected, or handled incorrectly.	Medium	High	High
R-03	Smart contract logic or deployment errors may prevent transfers or produce unreliable transaction status.	Medium	High	High
R-04	Frontend, backend, smart contract and MongoDB may not synchronize transaction hash, sender, receiver, amount and status correctly.	High	High	Critical
R-05	Weak authentication, RBAC or input validation may allow unauthorized access, invalid transfers or admin misuse.	Medium	High	High
R-06	MongoDB transaction history may not match blockchain results because of failed updates, duplicate submits or incomplete reconciliation.	Medium	High	High
R-07	Testing may miss negative cases such as invalid wallet address, insufficient balance, rejected signature, duplicate submit and empty fields.	High	Medium	High
R-08	Implementation may take longer than planned because blockchain, MetaMask and smart contract debugging require extra time.	High	Medium	High

7.6.3. Risk Response Table

Risk ID	Risk Description	Response Type	Response Action	Responsible Person
R-01	Requirements for wallet flow, confirmation or admin functions may change after implementation starts.	Mitigation	Confirm final scope with supervisor before implementation keep change log; approve only necessary changes.	Student / Supervisor
R-02	MetaMask connection or wallet ownership verification may fail when provider is missing, rejected or handled incorrectly.	Mitigation	Add wallet provider detection, approval/rejection handling and ownership-signature verification; test all wallet flows.	Student
R-03	Smart contract logic or deployment errors may prevent transfers or produce unreliable transaction status.	Mitigation	Test the smart contract on a local/test network; review transfer logic; record hash and failure reason.	Student
R-04	Frontend, backend, smart contract and MongoDB may not synchronize transaction hash, sender, receiver, amount and status correctly.	Mitigation	Define API and transaction status structure early; build an integration prototype; reconcile blockchain & MongoDB records.	Student
R-05	Weak authentication, RBAC or input validation may allow unauthorized access, invalid transfers or admin misuse.	Avoidance / Mitigation	Enforce protected routes, password hashing, RBAC, wallet address, amount, validation and unauthorized access tests.	Student
R-06	MongoDB transaction history may not match blockchain results because of failed updates, duplicate submits or incomplete reconciliation.	Mitigation	Use one transaction-record format, prevent duplicate submit, update status after execution and compare stored records with blockchain responses.	Student
R-07	Testing may miss negative cases such as invalid wallet address, insufficient balance, rejected signature, duplicate submit and empty fields.	Mitigation	Use test cases and add negative tests for invalid address, insufficient balance, rejected signature, duplicate submit and unauthorized history access.	Student
R-08	Implementation may take longer than planned for debugging smart contracts and MetaMask requires extra time.	Contingency Plan	Start blockchain integration early; keep a simpler demo flow as backup; reserve defect-fixing days before final submission.	Student

7.6.4. Risk Priority Table

Risk ID	Risk Description	Response Type	Response Action	Responsible Person
R-01	Requirements for wallet flow, confirmation or admin functions may change after implementation starts, causing rework.	Mitigation	Confirm final scope with supervisor before implementation; keep a change log; approve only necessary changes.	Student / Supervisor
R-02	MetaMask connection or wallet ownership verification may fail when provider is missing, rejected or handled incorrectly.	Mitigation	Add wallet-provider detection, approval/rejection handling and ownership-signature verification; test all wallet flows.	Student
R-03	Smart contract logic or deployment errors may prevent transfers or produce unreliable transaction status.	Mitigation	Test the smart contract on a local/test network; review transfer logic; record hash and failure reason.	Student
R-04	Frontend, backend, smart contract and MongoDB may not synchronize transaction hash, sender, receiver, amount and status correctly.	Mitigation	Define API and transaction status structure early; build an integration prototype; reconcile blockchain and MongoDB records.	Student
R-05	Weak authentication, RBAC or input validation may allow unauthorized access, invalid transfers or admin misuse.	Avoidance / Mitigation	Enforce protected routes, password hashing, RBAC, wallet/address/amount validation and unauthorized access tests.	Student
R-06	MongoDB transaction history may not match blockchain results because of failed updates, duplicate submits or incomplete reconciliation.	Mitigation	Use one transaction-record format, prevent duplicate submit, update status after execution and compare stored records with blockchain responses.	Student
R-07	Testing may miss negative cases such as invalid wallet address, insufficient balance, rejected signature, duplicate submit and empty fields.	Mitigation	Use test cases and add negative tests for invalid address, insufficient balance, rejected signature, duplicate submit and unauthorized history access.	Student
R-08	Implementation may take longer than planned for debugging smart contracts and MetaMask requires extra time.	Contingency Plan	Start blockchain integration early; keep a simpler demo flow as backup; reserve defect-fixing days before final submission.	Student

8. User Manual

8.1. Welcome Page

This is the welcoming page of the system, the user can either register or log in.

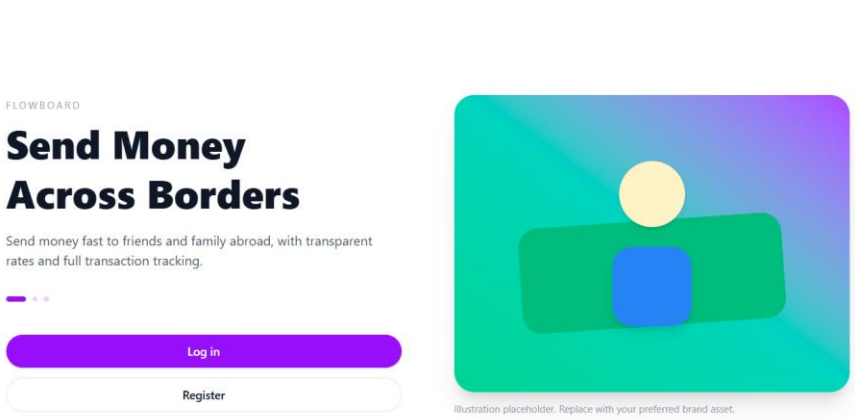


Figure 15: Welcome Page

8.2. Log in

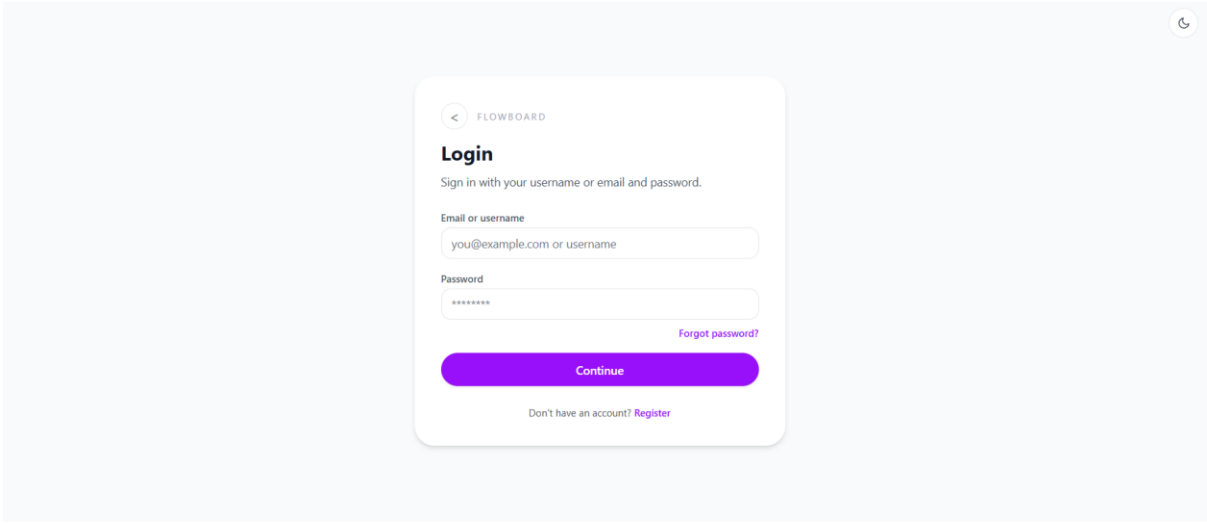


Figure 16: Log in page

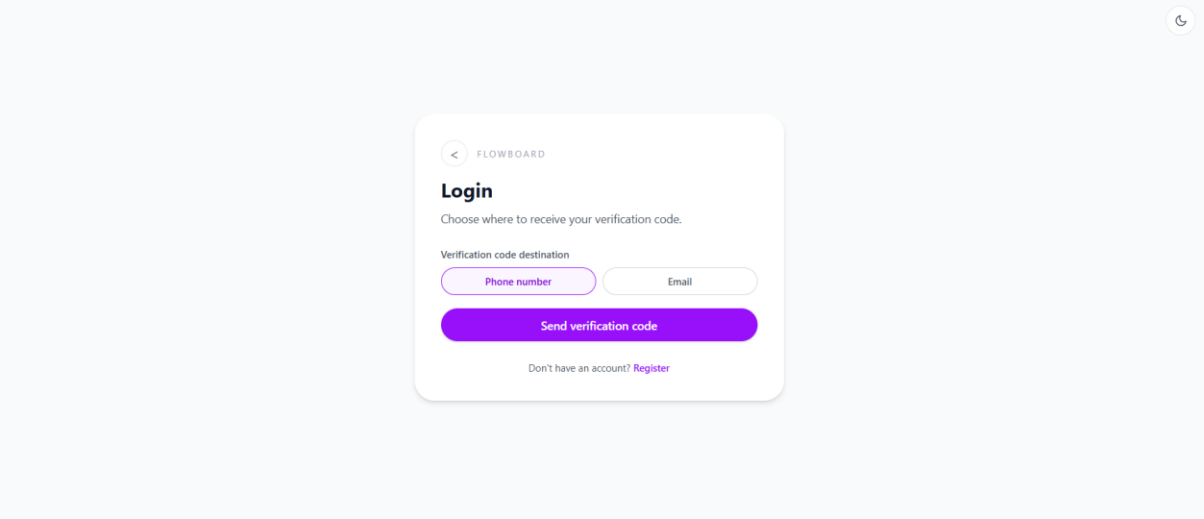


Figure 17: Send Verification Code

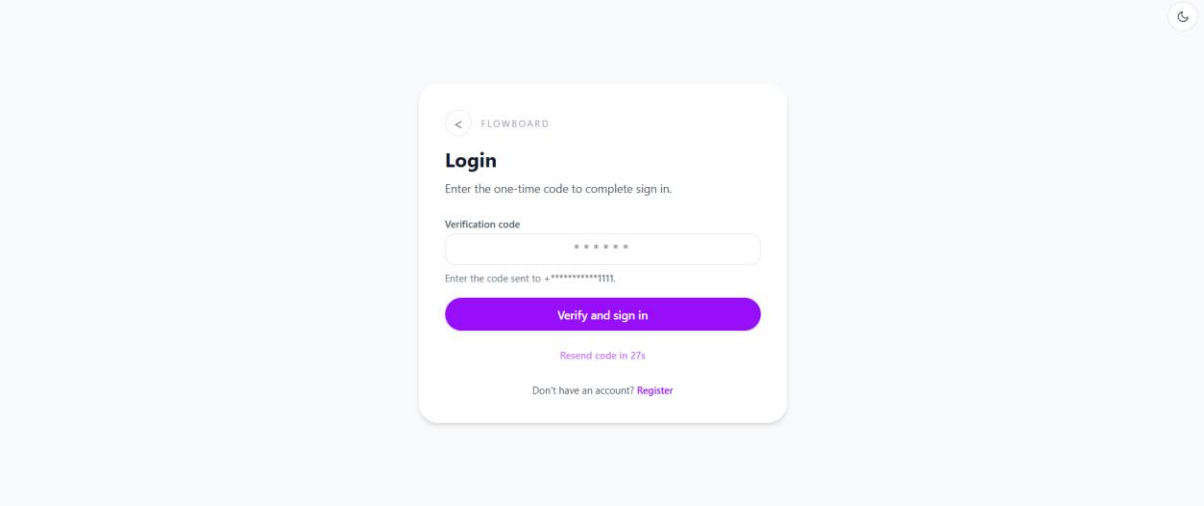


Figure 18: Verify and Log in

8.3. Dashboard

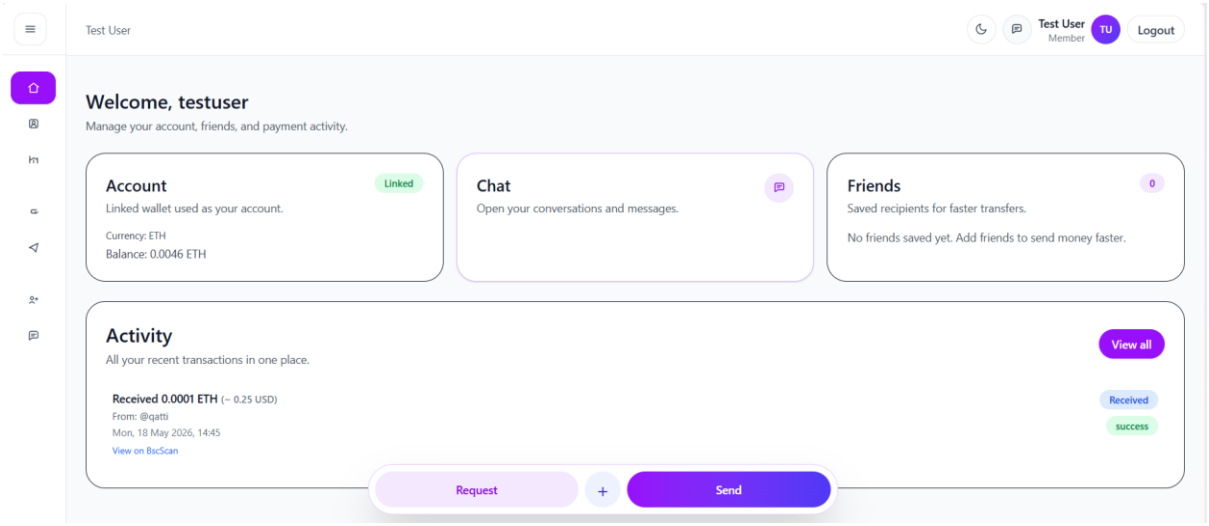


Figure 19: Dashboard Page

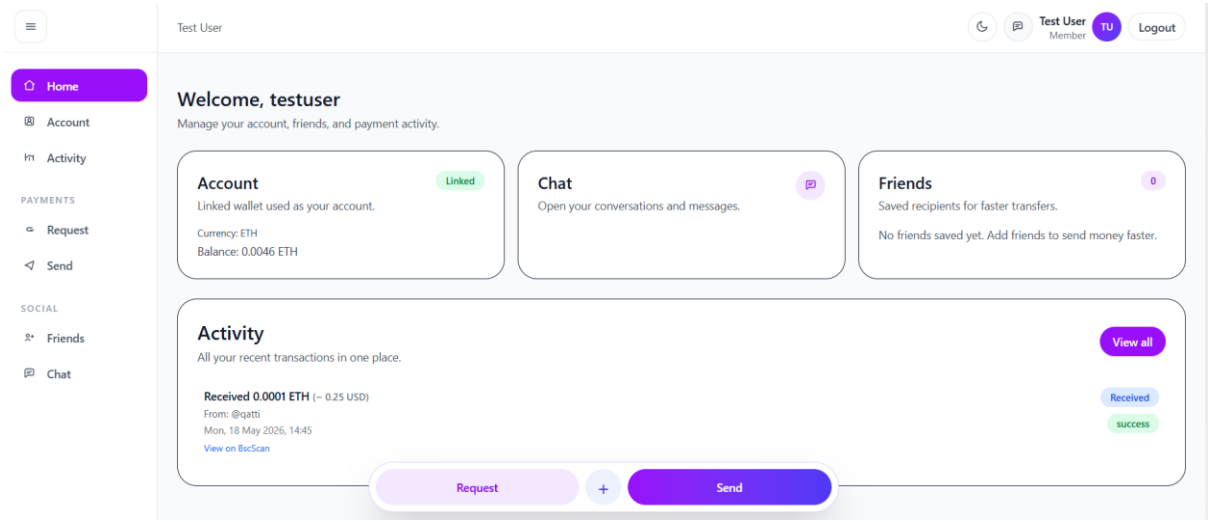


Figure 20: Dashboard with expanded sidebar

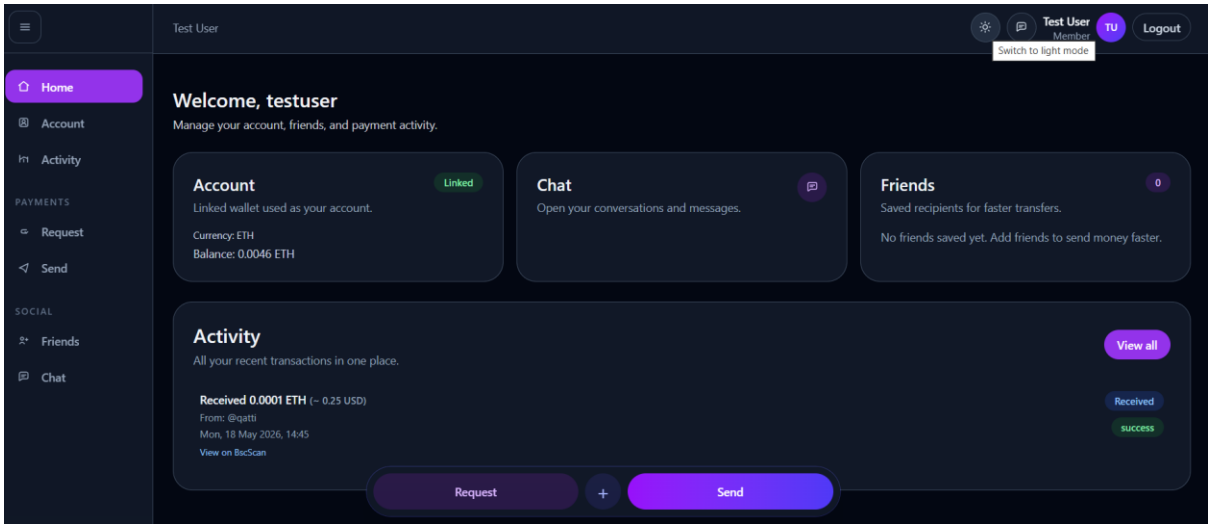


Figure 21: Switched to Dark Mode

8.4. Account Section

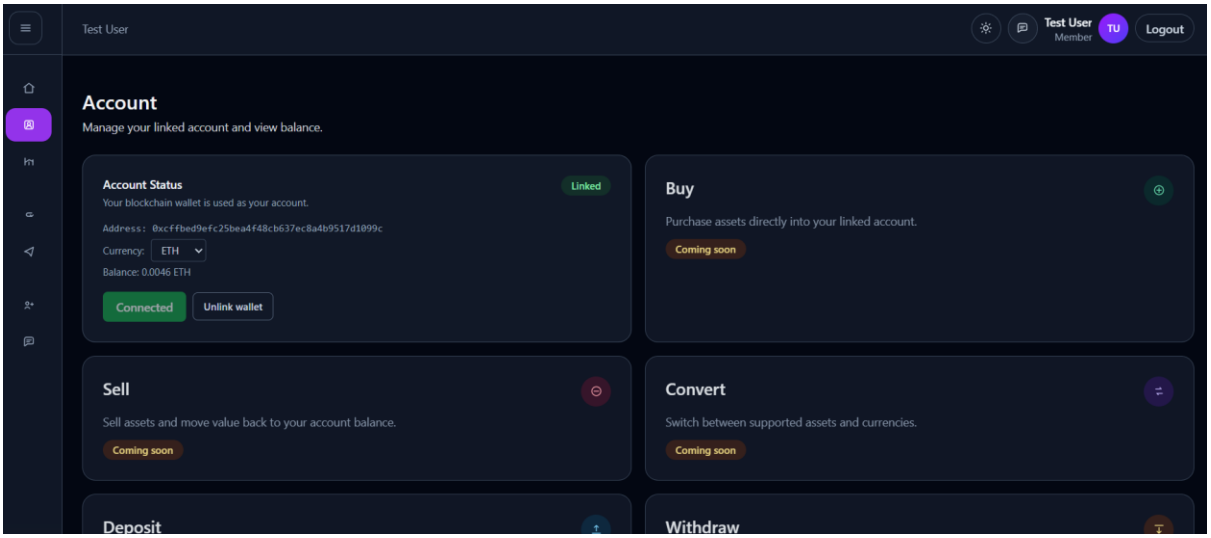


Figure 22: Account Page

8.5. Activities

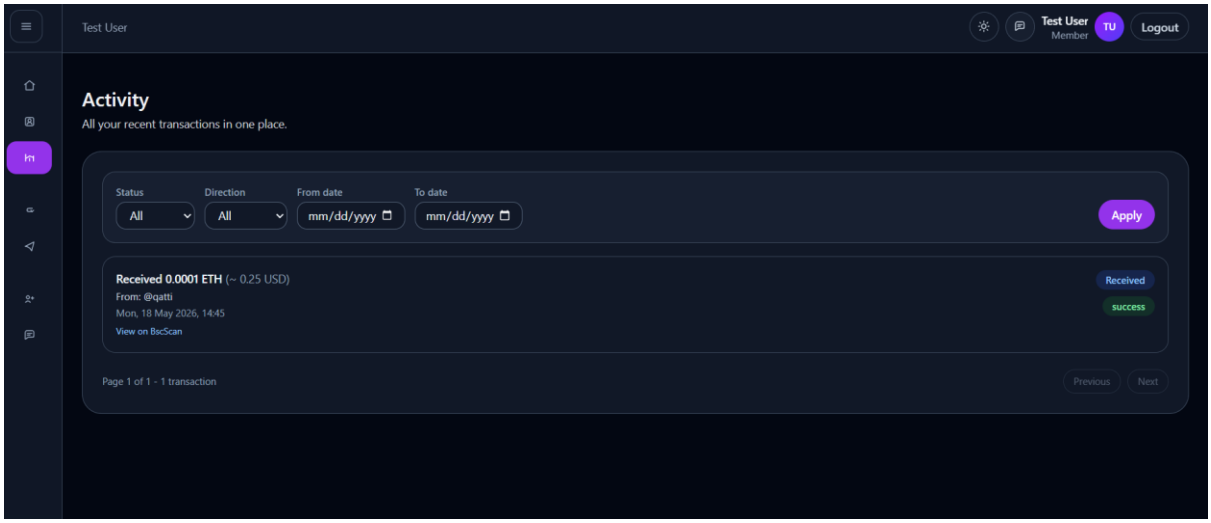


Figure 23: Activity Page

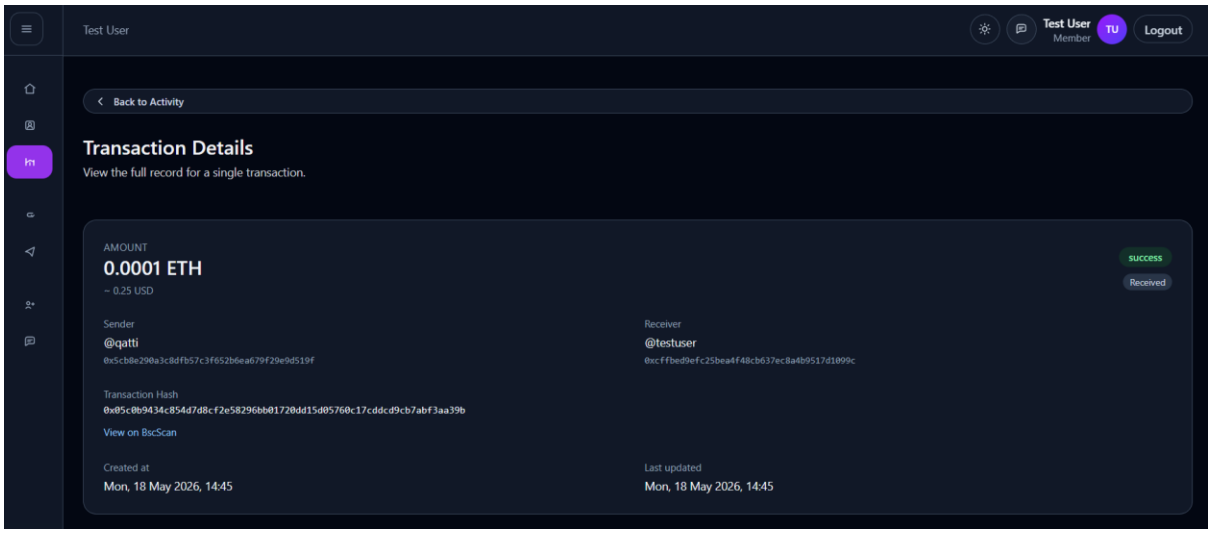


Figure 24: Transaction Details page

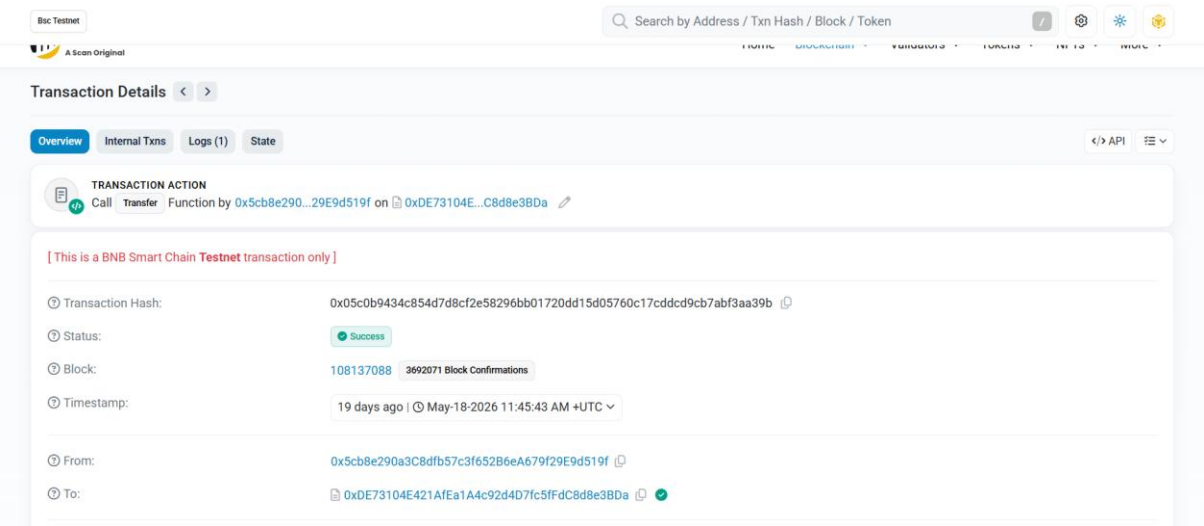


Figure 25: Transaction Details from External Website

8.6. Chat and Transfer to Friends

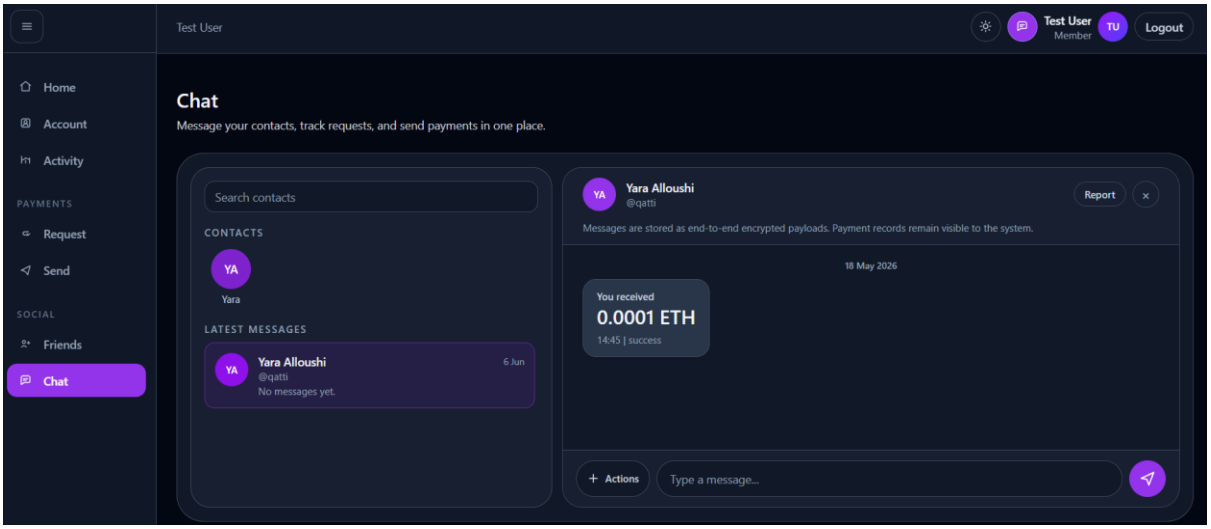


Figure 26: Chat with Friend

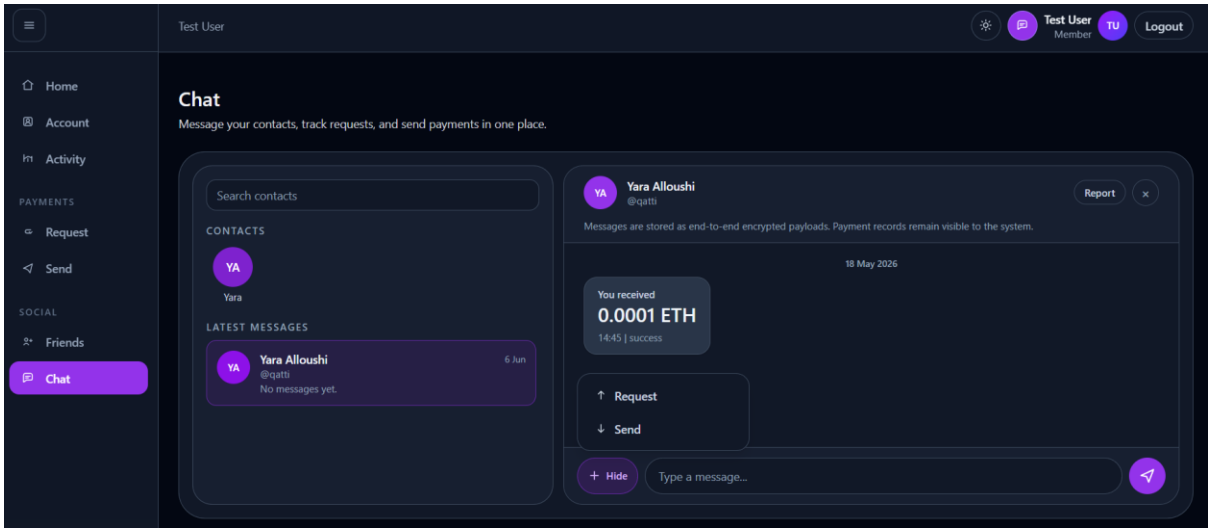


Figure 27: Option to Request and Send Money in Chat

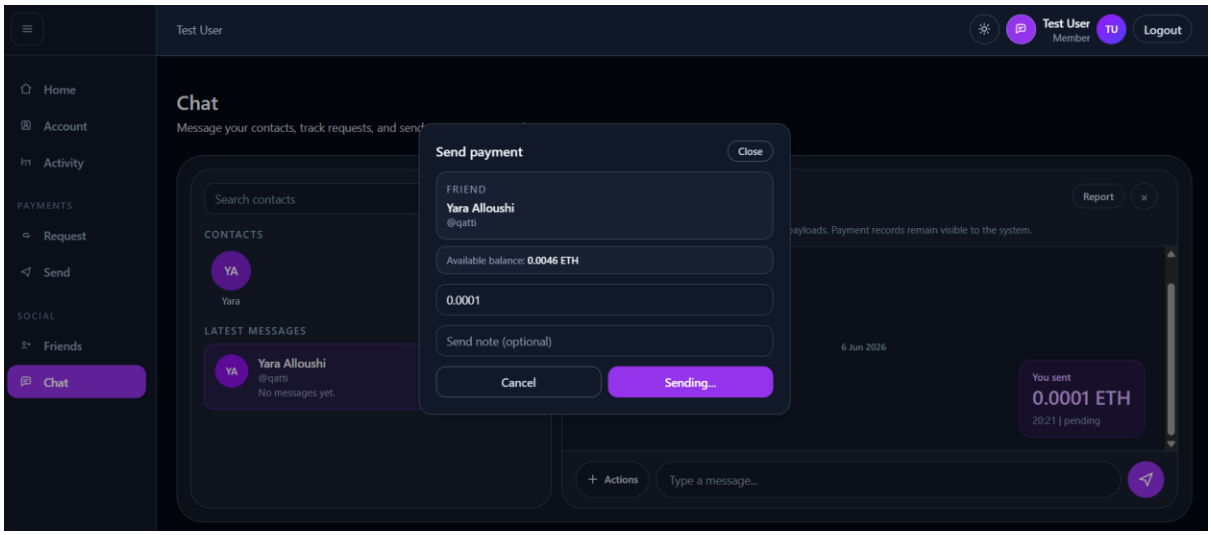


Figure 28: Sending 0.0001 ETH to a friend